CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Urban mobility has emerged as one of the defining challenges of modern cities. As urban areas expand outward through sprawling development and inward through population concentration, transportation systems are stretched beyond their original design limits. Increased ownership of private vehicles, the rapid growth of delivery services, and the heavy reliance on road transportation have produced traffic conditions that shift from predictable to highly variable throughout the day. Patterns now become irregular, with intersections frequently experiencing unpredictable congestion and significant slowdowns during busier periods. Even smaller municipalities formerly shielded from major traffic problems are beginning to face delays similar to those seen in large metropolitan regions.

Historically, traffic regulation was governed by relatively simple methods: fixed-interval traffic lights, occasional manual direction by traffic officers, and periodic adjustments based on on-site visual observations, they are no longer compatible with the complex and fast-paced mobility demands of today's urban environments. Fixed-timer signals operate on predetermined schedules, often forcing vehicles to stop even when the crossing lane has little or no traffic. During peak hours, these unchanging cycles fail to absorb increases in traffic density, causing long queues, slower travel, and rising frustration among commuters. Beyond reducing efficiency, poorly regulated traffic increases fuel consumption, elevates emissions, and accelerates the deterioration of roadway infrastructure.

The shift toward digital governance and the emergence of smart city strategies have reshaped expectations for traffic control. Advances in Internet of Things (IoT) devices, artificial intelligence, cloud analytics, and intelligent sensing technology now make it possible to construct adaptive and data-driven traffic management systems. A central component of this technological transformation is the IOT Based Emergency Priority Traffic System which analyses live traffic and adjusts signal timings automatically. In contrast to conventional signalling methods, an can recognize traffic buildup, detect abnormal patterns, update timing sequences in real time, and transmit system status to a centralized interface.

The need for intelligent the significance of systems becomes increasingly evident when considering emergency services. Ambulances, police units, and fire engines often face delays when navigating congested intersections, and even a short wait can influence the outcome of time-sensitive missions. Traditional traffic lights do not provide a structured way to prioritize these vehicles, resulting in slower and less predictable emergency responses. Additionally,

spontaneous disruptions such as road accidents, public events, seasonal congestion spikes, or maintenance work are difficult for manual or fixed systems to manage effectively.

This project utilizes these technological capabilities to develop a IOT Based Emergency Priority Traffic System designed to address present-day mobility challenges. The system incorporates continuous monitoring, adaptive signal control, and priority handling for emergency vehicles to create smoother and safer traffic conditions. By integrating IoT components with a central microcontroller, it processes real-time traffic data, modifies signal cycles accordingly, and displays operational information on an OLED screen. A web-based dashboard provides operators with remote access, allowing them to observe live conditions, adjust timing configurations, and activate emergency routes as needed. This combination of automated decision-making and human oversight offers a flexible, resilient, and practical approach for modern urban transportation.

1.2 PROBLEM STATEMENT

Urban transportation networks today are facing pressures that exceed the original purpose for which they were constructed. As cities expand, populations rise, and vehicle numbers continue to grow, daily traffic has become far more unpredictable and demanding. Delivery services and commercial transport add another layer of complexity, creating movement patterns that older infrastructures cannot efficiently support. Because of these combined factors, cities of all types now experience routine congestion that slows movement, disrupts schedules, and highlights weaknesses in current traffic control strategies.

One of the main contributors to this problem is the continued use of traffic lights that operate on fixed cycles. These systems run on preset schedules, assuming that traffic remains relatively steady throughout the day. In practice, however, traffic changes constantly due to work and school routines, weather conditions, accidents, public events, and unexpected disruptions. When signals cannot adjust to these fluctuations, intersections often operate inefficiently busy lanes endure long red phases while underutilized lanes receive more green time. The result is extended waiting periods, fuel wastage, and avoidable queuing.

Relying on manual oversight further complicates traffic management. Even experienced traffic officers can only observe and respond to so much at once. Their performance may be affected by fatigue, poor visibility, harsh weather, and the cognitive load of coordinating multiple lanes. During peak hours or sudden incidents, human supervision often becomes inconsistent, creating confusion and irregular traffic flow.

Another major issue lies in the limited use of real-time data. Many intersections still function without sensors, automated counting tools, or connected systems capable of tracking changing traffic patterns. Without continuous information, authorities cannot update signal timings appropriately or intervene before congestion escalates. This gap also affects emergency vehicles such as ambulances and fire trucks, which frequently struggle to navigate through stalled intersections, potentially delaying critical response times.

A final challenge is the difficulty of upgrading aging infrastructure. Many control systems implemented decades prior are not easily compatible with modern technologies, including sensors, microcontrollers, and adaptive algorithms. While such innovations could significantly improve traffic efficiency, smaller often lack the financial resources or technical expertise to implement them. This creates an uneven landscape where larger cities move ahead with advanced solutions while smaller communities remain dependent on outdated systems.

1.3 OBJECTIVES

- The main goal will be to create a smart traffic control system using ESP32 microcontroller, therefore demonstrating the possibility of using an adaptable controller to react to the current traffic instead of relying on outdated and fixed-in-time patterns.
- One of the main characteristics of the system is the inscription of an RC522 RFID reader that is able to differentiate between special or emergency vehicles; this characteristic allows the controller to adjust signal phases independently allowing preferential passage as necessary.
- The design incorporates the use of OLED display to present a small size and real-time visual interface, which displays the current signal conditions, read tags in RFID and priority selection and additional messages in a readable and friendly approach.
- A demonstrative model of a traffic intersection will be built using LED lights so that the project can display proper sequencing of lights, safe allowance intervals, and emergency override features in a way that is both demonstrative and easily understandable.
- ESP32 firmware is streamlined such that traffic processing, RFID interrogation and display updates can all run at once and allow all resources to communicate with each other in a smooth manor, thus there is little or no latency, or contention over resources.
- The work not only explains the drawbacks of the traditional traffic systems that were in use, i.e. unreliable operations, and strict schedule, but also shows how the automated control may produce more consistent and dependable results.

- The prototype will be implemented in a modular and scalable way, which will guarantee its interoperability with online-of-things solutions in future, its potential compatibility with many directions at a junction, and its adaptability to projects with smart cities.
- By introducing predictable sensor-controlled signal variations by strong timing principles, the total safety and traffic movement of road vehicle transportation becomes better, and this is a representation of the low-cost energy-saving, pedagogically valuable embodiments of Intelligent Transportation System principles.

1.4 SCOPE OF THE PROJECT

The scope of this IOT Based Emergency Priority Traffic System centres on designing a small-scale, hardware-driven model that illustrates how automated traffic control can be achieved using affordable embedded technologies. The project is limited to creating an intelligent intersection prototype in which an ESP32 microcontroller manages signal behaviour using data received from an RFID-based detection unit. At its core, the project aims to simulate the functioning of a standard traffic junction using three LEDs Red, Yellow, and Green to represent signal lights.

These LEDs are regulated entirely through ESP32 firmware, enabling the prototype to mimic real signal timings, orderly transitions, and appropriate delay intervals.The project scope also includes the use of an OLED display to provide continuous visual output regarding the system's operation. This display acts as a simple interface, presenting information such as the active signal phase, detected RFID tag data, and the activation of priority modes. Its inclusion makes the system easier to evaluate, monitor, and debug during prototyping. Another major aspect of the project involves developing robust ESP32 firmware capable of coordinating multiple tasks, such as handling input from the RFID reader, managing LED behaviour, verifying tags, executing timing logic, and updating the OLED screen.

The scope is further shaped by the intention to develop the prototype in a modular way, allowing additional components such as sensors, communication modules, or multi-intersection coordination to be added in the future without major redesign. Although the current project presents a simplified interpretation of traffic signalling, it provides a practical starting point for expanding into more advanced smart-city technologies, the scope remains intentionally limited: it does not include advanced elements such as video analytics, machine learning, cloud integration, image-based vehicle tracking, or deployment in real roadway environments. It also excludes regulatory components or vehicle-to-infrastructure communication frameworks. a microcontroller-based system can improve basic traffic control processes.

1.5. METHODOLOGY OVERVIEW

The methodology for IOT Based Emergency Priority Traffic System is built on coordinating several embedded components namely the ESP32 microcontroller, the RC522 RFID reader, an OLED display, and a set of LED indicators to create a functional model of an automated traffic junction. The process begins with initializing the ESP32, which acts as the central controller responsible for synchronizing all system operations. During startup, the microcontroller configures its GPIO pins for managing the Red, Yellow, and Green LEDs and establishes communication interfaces: SPI for interactions with the RFID module and I2C for the OLED display. At this stage, the firmware also loads the standard timing patterns for normal traffic flow and defines priority-specific responses for recognized RFID tags.

Once the system is initialized, it transitions into its continuous operating loop. In this state, the ESP32 persistently scans for RFID tags within the reader's range and captures the unique identifier of any detected card. The microcontroller compares this identifier with a predefined list of authorized IDs stored in memory. If the card is not recognized, the system maintains its regular signal sequence, cycling the LEDs according to preset time intervals. Throughout this process, the OLED display provides ongoing updates, showing the active light phase and any tag interactions to ensure transparent system behaviour.

When an authorized RFID tag is detected representing a priority or emergency vehicle the system temporarily shifts into a priority-handling mode. In this mode, the ESP32 overrides the standard traffic cycle and switches the LEDs to a safe configuration that grants immediate passage to the priority vehicle, typically by activating the Green signal for its direction. Safety delays and controlled transitions are incorporated to avoid abrupt signal changes. Meanwhile, the OLED display indicates that a priority event is in progress, allowing users to observe the system's response in real time. Once the priority condition ends, the ESP32 returns the system to its normal traffic sequence through a smooth transition.

Overall, this methodology focuses on maintaining safety, synchronization, and real-time responsiveness across all components. The structure of the firmware is designed to prevent conflicting signal states, ensure consistent communication among modules, and make the system adaptable for future expansions. Although implemented as a prototype, the approach supports scalability by allowing additional sensors, communication modules, or multi-junction features to be integrated without major architectural changes. The microcontroller compares this identifier with a predefined is In doing so, allowing the methodology demonstrates how embedded systems can effectively replicate intelligent traffic control in a manageable and reliable format.

1.6. PROJECT SIGNIFICANCE

The IOT Based Emergency Priority Traffic System developed in this project presents notable importance in responding to the increasing demands of modern traffic control. By combining ESP32-based automation with RFID-enabled priority detection and real-time feedback through both an OLED screen and a web dashboard, the system provides an updated and practical alternative to traditional traffic mechanisms. Instead of depending on static timers or manual supervision, the prototype introduces an adaptive, semi-automated model capable of adjusting to real-time conditions at an intersection.

A key contribution of the system is its capacity to recognize and prioritize emergency vehicles. Through RFID authentication, the system can instantly grant right-of-way to ambulances, fire services, and other critical responders, ensuring they move through intersections without unnecessary delays. This capability has the potential to shorten emergency response times and enhance public safety. Although developed on a small scale, the prototype effectively demonstrates how inexpensive IoT technologies can be leveraged to support smarter and safer urban mobility solutions.

The project also underscores the value of cost-effective innovation. The hardware used such as the ESP32 microcontroller, RC522 RFID reader, LEDs, and OLED display remains affordable and widely available, making the concept suitable for smaller cities, campuses, gated communities, and developing regions. The inclusion of a web-based interface further illustrates how remote monitoring and control can reduce reliance on human operators and help prevent manual errors.

The work is significant because it integrates multiple domains of engineering into a unified system. It strengthens understanding of microcontroller programming, embedded communication protocols like SPI and I2C, real-time decision- making logic, and user-interface design. The project also creates a starting point for future studies involving advanced features such as AI-based traffic optimization, cloud-coordinated signal control, or broader IoT-enabled transportation networks.

CHAPTER 2

LITERATURE REVIEW

2.1 ARDUINO-BASED EMERGENCY PRIORITY + STOLEN VEHICLE.

The study titled “Smart Traffic Management System with Emergency Vehicle Prioritization and Stolen Vehicle Detection Using Arduino Technology” (2024) by Abhirami J.S., Dinesh Kumar M., Prasanth S., and Dalbert Mario J. present an economical Arduino-based approach to improve responsiveness at traffic intersections. Their prototype automates routine signal control while giving precedence to emergency responders such as ambulances, fire engines, and police vehicles. As these vehicles approach a junction, the system processes incoming identification data and alters the light cycle to clear their path. Additionally, the design incorporates a feature that checks the status of passing vehicles, allowing detection of those reported stolen. The authors emphasize that Arduino boards, while suitable for small-scale applications, may face limitations when deployed across large, high-density traffic networks. To overcome these constraints, they employed optimized real-time processing logic to enhance adaptability during sudden or unpredictable changes in traffic flow.

2.2 EDGE + IOT METHOD FOR FAST EMERGENCY CLEARANCE.

In “An Optimal Control Strategy for Emergency Vehicle Priority System in Smart Cities Using Edge Computing and IoT Sensors” (2023), R. Pradeep, M. Ramesh, P. Suresh, and collaborators propose a more technologically advanced smart-city framework. Emergency vehicles equipped with GPS-enabled IoT modules continuously transmit their coordinates to nearby edge-computing nodes. These edge devices analyse the incoming data locally rather than relying on distant cloud servers, allowing traffic lights across multiple intersections to adjust rapidly. Although this method significantly reduces system delay, the researchers acknowledge that abrupt priority changes can disrupt the normal behaviour of surrounding drivers. To address this, they introduce an optimization strategy that moderates signal transitions, reducing unnecessary stops and helping maintain smoother traffic continuity.

2.3 IOT SETUP TO PRIORITIZE EMERGENCY VEHICLES

The third work, “IoT-Based Traffic Management System Prioritizing Emergency Vehicles” (2023) by A. Sharma, R. Kumar, and S. Gupta, presents an IoT-integrated model capable of assessing congestion levels and modifying signal timing automatically. A major feature of this system is a remote web-based override that allows traffic officials to intervene

manually whenever needed. Arduino hardware is used to measure real-time congestion, while an online dashboard provides a live view of conditions at each junction. However, the authors point out that the system's manual controls depend heavily on a stable internet connection. Weak or intermittent connectivity may cause delays in executing remote commands. To improve reliability, they combine automated signal logic with manual fallback options so the system remains operational even during network fluctuations.

2.4 RFID TO DETECT EMERGENCY AND THEFT VEHICLES.

The study "IoT-Based Emergency and Theft Vehicle Identification in Traffic System Using RFID" (2024) by Velmurugan V., Arunkumar B., Deepika B., Dhanasree M., and Kumaram S. centers on implementing RFID technology for identifying emergency and stolen vehicles. Emergency responders carry authorized RFID tags instantly recognize their presence and turn successive signals green along their route. Vehicles flagged as stolen generate alerts each time they pass through an RFID-equipped junction. Although the RFID approach offers rapid and non-contact identification, the authors note drawbacks such as limited reading distance and potential interference from environmental or electronic sources. They address these issues by adjusting antenna placement and incorporating supplementary sensors to strengthen detection accuracy at intersections.

2.5 RFID-BASED SIGNAL PRIORITY FOR EMERGENCY VEHICLES.

The final study, "RFID-Based Traffic Control System for Emergency Vehicle" (2023) by Gowtham M., Adhwanth D., Manojkumar G., Naveen R., and Nazeem K.I., also utilizes RFID technology to streamline the movement of emergency vehicles. When an authorized vehicle approaches, its tag is detected and the system reorganizes the traffic signal to favour its direction of travel. The authors highlight a key challenge: integrating RFID capability with existing traffic-light infrastructures while still maintaining real-time reaction speed. Using Arduino UNO boards and optimized control algorithms, they successfully enhanced system responsiveness and ensured reliable operation during sudden shifts in traffic demand. The authors also highlight the operational challenges involved in deploying RFID solutions in real-world traffic networks. These include ensuring proper placement of readers to avoid blind zones, preventing interference from nearby electronic equipment, and maintaining compatibility with existing traffic controllers. This reinforces the idea that even in low-power embedded systems, emergency signal management can be highly efficient.

2.6 LITERATURE SURVEY SUMMARY TABLE

Sl. No	Title of the Paper	Year	Authors	Technical Ideas	Shortcomings & How It's Addressed
1	Smart Traffic Management System with Emergency Vehicle Prioritization and Stolen Vehicle Detection Using Arduino Technology	2024	Abhirami Js, Dinesh Kumar M, Prasanth S, Dalbert Mario.J	Developed an Arduino-based system integrating traffic signal control with automatic clearance for emergency vehicles and stolen vehicle detection. Utilizes real-time data processing to adjust signal timings dynamically.	Potential limitations in scalability and adaptability to varying traffic conditions. Addressed by implementing real-time data processing and decision-making algorithms.
2	An Optimal Control Strategy for Emergency Vehicle Priority System in Smart Cities Using Edge Computing and IoT Sensors	2023	R. Pradeep, M. Ramesh, P. Suresh, et al.	Proposed an optimal control strategy employing edge computing and IoT sensors to prioritize emergency vehicles. Utilizes GPS-based IoT sensors to send location information to edge servers, which compute optimal timings.	Sudden stoppage of normal traffic causing confusion. Addressed by implementing an optimal control strategy that minimizes disruption to normal traffic flow.
3	IoT-Based Traffic Management System Prioritizing	2023	A. Sharma, R. Kumar, S. Gupta	Presented an automated traffic signal monitoring system based on IoT, allowing manual override over the	Dependence on internet connectivity for manual override. Addressed by incorporating both

	Emergency Vehicles			internet. Controls traffic signal density using an Arduino-based circuit system and displays current densities via an online GUI.	automated and manual control mechanisms to ensure reliability.
4	IoT-Based Emergency and Theft Vehicle Identification in Traffic System Using RFID	2024	Velmurugan V., Arunkumar B., Deepika B., Dhanasree M., Kumaram S.	Developed a system providing clearance to emergency vehicles by turning all red lights to green along their path. Also tracks stolen vehicles passing through traffic lights using RFID technology.	Limited range and potential interference of RFID signals. Addressed by optimizing antenna placement and integrating additional sensors for improved detection.
5	RFID-Based Traffic Control System for Emergency Vehicle	2023	Gowtham M., Adhwanth D., Manojkumar G., Naveen R., Nazeem K. I	Implemented a smart traffic control system using RFID sensors to identify emergency vehicles and adjust traffic signals accordingly. Aims to reduce congestion and improve emergency response times.	Challenges in real-time data processing and integration with existing infrastructure. Addressed by utilizing Arduino UNO and efficient programming for timely responses.

Table 2.1 Summary of Literature Review

CHAPTER 3

OVERVIEW OF PROPOSED METHODOLOGY

3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed IOT-Based Emergency Priority Traffic System is designed to modernize conventional signal operations by incorporating automation, remote supervision, and intelligent handling of emergency situations into a single unified platform. Traditional traffic signals generally rely on predetermined timing schedules or require manual adjustments at the intersection approaches that fall short when traffic density changes unexpectedly. The system developed in this project replaces those limitations with an ESP32 microcontroller that oversees signal coordination, hosts an online control dashboard, and provides real-time status updates through an OLED display. This transforms the traffic signal into a flexible, responsive, and easily accessible setup that can be monitored or controlled from any authorized device.

Functionally, that manages a three-direction junction North, East, and South each direction equipped with its own Red, Yellow, and Green LEDs. The ESP32 acts both as the logic controller and as a built-in web server. Through a standard Authorized users can access the web browser. access an interactive dashboard showing which direction currently holds the green signal, the countdown for each phase, and options for adjusting signal durations instantly. These dynamic modifications allow the system to respond to changing conditions without relying on fixed-cycle timing or field personnel.

A core capability of the system is its Emergency Vehicle Priority Mode. This feature ensures that ambulances and other priority vehicles receive immediate right-of-way at the intersection. When activated, the system pauses the normal cycle, sets the target direction to green, and switches all other approaches to red. A blue LED indicates that priority mode is active, while the OLED display shows relevant details in real time. This system guarantees that emergency vehicles can pass without obstruction, helping reduce critical response delays.

3.2 SYSTEM ARCHITECTURE

The IOT-Based Emergency Priority Traffic System introduced in this project offers an efficient and contemporary upgrade to traditional intersection control practices. Instead of relying on rigid timing schedules or requiring human operators to adjust signals on-site, the system uses an ESP32 microcontroller to automate light sequences, support remote access, and manage emergency vehicle passage intelligently. Through IoT connectivity and a browser-accessible interface, traffic personnel can observe intersection activity in real time and modify signal behaviour with improved accuracy and responsiveness.

At the centre of the system’s framework, the ESP32 acts both as the main processing unit and an embedded web server. It manages the signal transitions for three roadway directions

North, East, and South each equipped with red, yellow, and green LEDs. Using Wi-Fi, authorized users can open the dashboard to review active signals, change timing settings, or immediately trigger the emergency-priority function when needed. This remote management capability eliminates the reliance on physical operators at the intersection and lets the system to respond quickly to varying traffic conditions.

One of the system's most important features is its Emergency Vehicle Priority Mechanism. When an ambulance or any designated priority vehicle approaches, the system can provide a direct route by adjusting the signal sequence. This priority mode can be triggered automatically when the RFID module detects a registered emergency through a dedicated dashboard control. Once activated, the ESP32 halts the regular cycle, enables a green for the required direction, sets all other directions to red, and turns on a blue priority indicator.

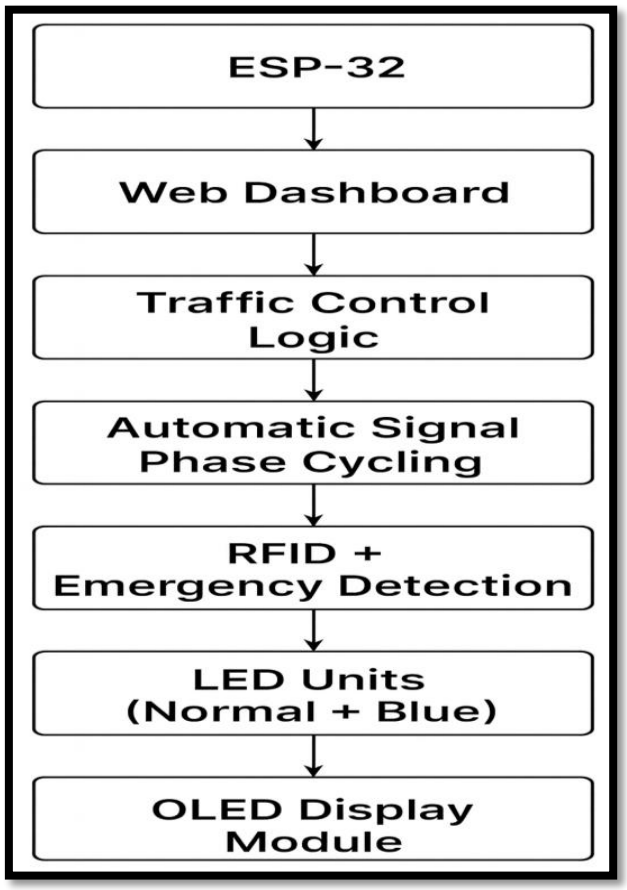

Fig 3.1 System Architecture Diagram

In Figure 3.1, an ESP-32–based smart traffic system manages signal control through a web interface. It supports RFID-based emergency prioritization with visual output displays.

3.3 MODULES DESCRIPTION

3.3.1 ESP32 TRAFFIC SIGNAL CONTROL UNIT

This module serves as the principal control centre of the system. The ESP32 microcontroller governs the automated traffic-light sequence for the North, East, and South

lanes. It uses a non-blocking timing strategy to manage Green, Yellow, and Red intervals, while a structured state machine ensures that only one direction receives the active signal at a time. This prevents conflicting outputs and maintains safe transitions throughout the cycle. The module also works with the other subsystems to adjust timing settings, respond to emergency triggers, and broadcast status updates to all connected display and communication interfaces.

3.3.2 WEB DASHBOARD AND IOT CONNECTIVITY UNIT

This module provides remote access and real-time interaction through a web-based interface. Acting as a Wi-Fi server, the ESP32 hosts a built-in webpage where users can view the active lane, modify timing parameters, enable emergency priority mode, or reset the system entirely. Communication relies on lightweight HTTP exchanges, making the dashboard responsive and accessible from any device connected to the same network. No dedicated software is required, enabling convenient and flexible control from laptops, tablets, or smartphones.

3.3.3 RFID-ENABLED EMERGENCY VEHICLE PRIORITY UNIT

This module detects and prioritizes emergency vehicles approaching the intersection. The RFID reader identifies an authorized tag carried by an ambulance, fire truck, or other emergency unit. When a recognized tag is detected, the system immediately overrides the standard traffic cycle, activating a Green light for the relevant direction while forcing all other directions to Red. This automatic, high-speed response removes the need for manual intervention and ensures that emergency vehicles pass through the intersection without unnecessary delay. It operates in coordination with the signal controller and display interface.

3.3.4 OLED REAL-TIME MONITORING AND DISPLAY UNIT

This module provides immediate on-device visual feedback regarding the system's operation. The OLED screen displays the current active direction, signal colour, time remaining in the current phase, and the status of any emergency override. Whenever the system updates either automatically or When a recognized through dashboard input the display refreshes accordingly. the system immediately overrides the standard traffic cycle, This module is especially on-device visual feedback valuable for live demonstrations, and troubleshooting, or field use, then where quick and the status interpretation is needed without relying on the web dashboard.

3.4 TECHNOLOGIES AND TOOLS USED

3.4.1 ESP32 MICROCONTROLLER

The ESP32 functions as the central processing unit of the entire traffic system. It is responsible for executing the traffic-light sequence, handling phase transitions, responding to emergency triggers, driving LED indicators, and updating the OLED display. One of its major

strengths is the built-in Wi-Fi capability, which allows it to host a web dashboard without requiring additional networking hardware. Its dual-core architecture, low power consumption, and extensive GPIO availability make it highly suitable for long-duration IoT applications, especially systems that demand multitasking and constant connectivity.

3.4.2 RFID READER AND EMERGENCY TAG MODULE

Emergency vehicle detection is implemented using an RFID reader–tag configuration. The MFRC522 module identifies predefined RFID tags mounted on emergency vehicles and triggers immediate signal priority when a valid tag is scanned. This method provides a rapid, contactless detection mechanism and eliminates reliance on manual overrides. The module is affordable, easy to interface using SPI communication, and proven effective for prototype-level emergency response systems.

3.4.3 LED-BASED TRAFFIC SIGNAL INDICATORS

Traffic signalling is represented using sets of red, yellow, and green LEDs assigned to each direction, along with a dedicated blue LED for indicating emergency mode. These LEDs emulate real-world signal behaviour while consuming minimal power. They integrate seamlessly with the ESP32's GPIO pins and provide fast, accurate visual transitions. Their simplicity and reliability make them ideal for microcontroller-based traffic control models.

3.4.4 OLED DISPLAY UNIT

A compact OLED screen is used to present real-time system information such as the current direction, signal state, countdown timer, and emergency alerts. Due to its high contrast, low energy consumption, and easy I²C interfacing, the OLED is highly effective for on-site monitoring during demonstrations and testing. It enhances system transparency and allows users to understand system activity without accessing the web dashboard.

3.4.5 SUPPORTING ELECTRONIC COMPONENTS

The system is built on standard electronic components including jumper wires, resistors, connectors, breadboards, and regulated USB power supply units. enable emergency priority mode These components ensure safe wiring, stable power distribution, and flexibility during prototype development. They also make modifications and troubleshooting more manageable during testing phases.

3.4.6 ARDUINO IDE

Firmware development for the ESP32 is carried out using the Arduino IDE. This environment simplifies code writing, compilation, and uploading while offering extensive library support for Wi-Fi connectivity, RFID communication, OLED display updates, and time-based control. Its user-friendly interface and strong ecosystem make it an ideal choice for ESP32-based IoT projects.

3.4.7 ESP32 WI-FI AND NETWORKING LIBRARIES

The system relies on built-in Wi-Fi and networking libraries that allow the ESP32 to function as a web server and manage HTTP-based communication. These libraries support real-time data exchange with any browser-equipped device and enable users to monitor or adjust system parameters wirelessly. The lightweight nature of HTTP requests ensures fast and efficient dashboard interaction.

3.4.8 EMBEDDED WEB DASHBOARD (HTML, CSS, JAVASCRIPT)

The user interface is implemented as a browser-based dashboard developed using HTML, CSS, and JavaScript. Stored within the ESP32's memory, it allows users to modify signal timings, enable emergency priority, and observe live traffic conditions. Because it operates entirely over local Wi-Fi, no internet connection or external hosting is required. Its simplicity and responsiveness make it accessible on any platform.

3.4.9 RFID COMMUNICATION LIBRARIES

Libraries such as MFRC522.h are used to manage RFID scanning, tag authentication, and emergency-priority activation. These libraries abstract low-level SPI communication, making RFID integration straightforward and reliable. They ensure accurate performance during continuous scanning for emergency vehicles.

3.4.10 OLED DISPLAY DRIVER LIBRARIES

Display libraries including Adafruit GFX and SSD1306 support smooth rendering of text and system information on the OLED module. Their non-blocking update functions enable the screen to refresh without interrupting the main traffic logic or emergency routines, ensuring consistent and real-time display performance.

3.4.11 HTTP-BASED COMMUNICATION PROTOCOL

The system uses lightweight HTTP GET requests to handle dashboard interactions such as timing updates, emergency activation, and resets. This protocol enables direct and efficient communication between the ESP32 and any web browser without requiring additional cloud services or MQTT brokers. Its low latency ensures rapid feedback and improved user experience.

3.5 ADVANTAGES OF THE PROPOSED SYSTEM

The IoT-supported traffic management system suggested has several strengths as compared to the traditional fixed-time traffic signals. It has one of its main advantages in the ability to provide a flexible and smart signal control with the possibility of the traffic-light phases automatically altering rather than following strict time schedules. This translates into an efficient and responsive car traffic flow especially during different traffic conditions.

One of the most assumed advantages of the system is the fact that it prioritizes emergency vehicles at once. RFID based detection can be used to identify ambulances and all emergency units instantaneously and therefore the system can issue a green signal without delay in waiting the normal cycle. This aspect considerably minimizes the response times and increases road safety.

Remote control is also baked into the system using a web-based dashboard which allows operators to see the real time status and edit signal times through any device with Wi-Fi. This eliminates physical adjustments in the junction and allows control flexibility and control on command. The dashboard is implemented in the form of an OLED screen to show permanent information on the device, such as the direction of movement, the state of a signal, a countdown clock, and emergency warnings.

Performance The system has non-blocking behavior and a smooth operation since timing is handled and not delay functions. As a result, continuous multitasking and extreme responsiveness is obtained. Moreover, the system increases the safety at the intersection points, because there are no conflicting signals at the intersection and the path of emergency vehicles is always clear and without any obstructions in serious situations.

Operationally, the design will result in reduced congestion and vehicle waiting time due to reduced unnecessary stops and encouragement of smooth traffic flow. It is also easy to implement and saleable, constructed on the ESP32 platform, which is cheap, low-power consuming, and able to expand to support expansion in the future to more intersections.

The system can greatly reduce the presence of manual involvement. The use of automated control, handling of emergencies and inbuilt displays enhance ease in traffic control, and easy diagnostics and maintenance. Live feedback provided through LEDs, OLED display, and dashboard analytics make troubleshooting faster, which guarantees the long-term working durability.

CHAPTER 4

REQUIREMENTS SPECIFICATION

4.1 FUNCTIONAL REQUIREMENTS

4.1.1 AUTOMATIC TRAFFIC SIGNAL CONTROL

The system is designed to operate the traffic lights for the North, East, and South directions autonomously, following a structured and safe signal sequence. The ESP32 manages the transition from Green to Yellow and then to Red before shifting to the next direction. At any given moment, only one direction is permitted to display either a Green or Yellow signal, ensuring that conflicting traffic movements never occur. All other directions stay in the Red state while one side is active. This automated cycle runs continuously without requiring human input, providing smooth and accurately timed transitions to maintain predictable traffic flow throughout operation.

4.1.2 CUSTOMIZABLE SIGNAL TIMINGS VIA WEB DASHBOARD

The system supports real-time modification of traffic-light timings through a web dashboard. Users can adjust the Green, Yellow, and Red durations individually for each direction from any device connected via Wi-Fi, such as a phone, laptop, or tablet. These changes are applied immediately to the ongoing cycle without requiring a restart of the ESP32 controller. This flexibility allows operators to adapt the signal timings to varying traffic conditions and special events quickly and efficiently.

4.1.3 REAL-TIME WEB DASHBOARD DISPLAY

A live, browser-based interface provides continuous visibility of the system's operational state. The dashboard shows which direction currently has the Green light, the active signal colour for each lane, and the remaining time for the current phase. It also offers interactive options for adjusting timings, activating emergency mode, and resetting the controller. The display refreshes dynamically, enabling users to monitor traffic operations with precision and clarity.

4.1.4 EMERGENCY VEHICLE PRIORITY (RFID + DASHBOARD TRIGGER)

The system incorporates an emergency-priority mechanism to immediately clear the path for ambulances and other emergency responders. Priority can be triggered either by scanning an authorized RFID tag or by manually selecting an emergency option through the dashboard. Once activated, the chosen direction turns Green instantly, while all other directions are forced to Red. A blue indicator light switches on to signal emergency mode, and the normal cycle is temporarily paused. After the emergency vehicle has passed, the system returns to the automated cycle at the appropriate phase without requiring any manual adjustment.

4.1.5 RFID-BASED EMERGENCY AUTHENTICATION

The system uses RFID technology to accurately identify emergency vehicles. When the RFID reader detects a tag, the ESP32 verifies whether the ID matches a pre-registered emergency identifier. If the tag is valid, emergency-priority mode is activated immediately. Any

unrecognized or invalid tag is ignored to ensure system security and prevent accidental priority activation.

4.1.6 OLED STATUS DISPLAY

The OLED module provides clear, real-time feedback directly on the hardware. It displays the active direction, the signal colour, and the remaining time for the current phase. During emergency situations, it shows an alert indicating that priority mode is active. The display also presents initialization messages when the system starts, helping users understand system status instantly even without accessing the dashboard.

4.1.7 IOT CONNECTIVITY AND COMMUNICATION

The ESP32 supports wireless communication through Wi-Fi, enabling seamless interaction with the dashboard. It runs a web server that responds to various HTTP endpoints for functions such as loading the interface, sending live status updates, adjusting signal timings, activating emergency modes, other emergency responders and restoring the system to automatic operation. This communication structure ensures fast, reliable, and real-time control from any browser-enabled device.

4.1.8 MANUAL OVERRIDE CONTROLS

In addition to automatic operation, the system provides manual override options through the dashboard. Users can pause the automatic cycle, reset the controller, trigger emergency mode for a specific lane, or apply timing updates across all directions simultaneously. These features are especially useful during maintenance, testing, or unusual traffic conditions that require immediate human intervention.

4.1.9 CONTINUOUS AND STABLE OPERATION

The controller is intended for long-duration use, maintaining stable performance without interruptions. By using non-blocking timing functions, the ESP32 avoids freezing and continues updating all outputs smoothly. to the ongoing cycle without requiring a restart .Signal transitions remain consistent and conflict-free, and the system automatically returns to normal cycling once an emergency override ends. This ensures dependable and uninterrupted operation in real-world scenarios.

4.1.10 SAFETY AND FAIL-SAFE HANDLING

Safety is built into the system through multiple fail-safe mechanisms. The controller rejects any invalid, unsafe, or negative timing values entered through the dashboard. Malformed HTTP requests are ignored to prevent system MISCONFIGURATION. If an improper input is detected, the system continues functioning using the last known valid settings. Additionally, the logic guarantees that no two directions display Green or Yellow simultaneously, maintaining

traffic safety at all times. Warning messages or error notifications are shown through the dashboard or OLED whenever necessary.

4.2NON-FUNCTIONAL REQUIREMENTS

4.2.1 RELIABILITY AND SYSTEM AVAILABILITY

The system is designed to operate continuously without unexpected interruptions, ensuring that the traffic-light controller remains active at all times. Even if Wi-Fi connectivity is temporarily lost, the core traffic-signal logic continues functioning normally, preventing disruptions at the intersection. Once the network connection is restored, all dashboard-dependent features automatically resume without requiring a restart. This ensures high availability and dependable operation, especially during long-duration demonstrations.

4.2.2 PERFORMANCE AND RESPONSE EFFICIENCY

High responsiveness is essential for an IoT-based control system. The controller is expected to process dashboard commands such as timing changes, emergency activation, or reset operations within a very short time, generally under one second. Signal transitions must occur exactly according to the configured durations, maintaining strict timing accuracy. The OLED display should also reflect every phase update instantly, allowing users to observe real-time system behaviour without delays.

4.2.3 SCALABILITY AND EXPANDABILITY

The system architecture is intentionally designed to allow future expansion. Additional directions or lanes can be integrated with minimal modifications to the codebase. The same design principles also make it feasible to extend the system into a network of multiple interconnected junctions. New hardware components, such as extra RFID readers, sensors, or display units, can be incorporated without major restructuring, supporting long-term scalability.

4.2.4 USABILITY AND INTERFACE QUALITY

The web dashboard serves as the primary user interface and must remain easy for non-technical users to understand and operate. All major functions including timing adjustments, emergency control options, and system reset are arranged in a clear manner so users can locate them quickly. The OLED display further enhances usability by presenting concise and readable status information directly on the hardware, making the system accessible even without opening the dashboard.

4.2.5 MAINTAINABILITY AND MODULAR DESIGN

The entire system is developed with modularity in mind, allowing various subsystems such as the firmware, dashboard interface, traffic LEDs, or RFID reader to be serviced or upgraded independently. The firmware follows a structured coding approach that simplifies

debugging and future enhancements. Modifications to timing rules, interface layouts, or display logic can be implemented without affecting the core system logic, ensuring easy maintenance and long-term sustainability.

4.2.6 SECURITY AND ACCESS CONTROL

Secure operation is a critical requirement, especially since the system influences traffic flow. Only authorized users should be able to access the dashboard and issue control commands. Any malformed or suspicious requests must be ignored to prevent manipulation or accidental disruption. Sensitive operations such as emergency overrides or signal-timing adjustments must remain protected to ensure safe and controlled usage.

4.2.7 ENERGY EFFICIENCY

The system is expected to function efficiently with minimal power consumption. The ESP32's low-power capabilities should be utilized wherever possible, ensuring extended operation during demonstrations or portable use. The RYG LEDs and OLED display must operate within safe power limits, preventing overheating and unnecessary energy waste. This makes the prototype suitable for long-term educational and laboratory use.

4.2.8 ROBUSTNESS AND FAULT TOLERANCE

The design emphasizes fault-tolerant behaviour to maintain safe traffic control even under abnormal conditions. If a user enters invalid or unsafe timing values, the system should automatically revert to the last correct configuration rather than ignored applying faulty settings. The ESP32 must also continue operating even when a peripheral such as the OLED or RFID temporarily malfunctions. an IoT-based control system generally power Conflicting developed green signals or simultaneous unsafe outputs must never occur under any circumstance.

4.2.9 COMPATIBILITY AND PLATFORM INDEPENDENCE

The dashboard is built to function across a wide range of modern web browsers, including those on Android and iOS devices. No external software or app installation is required, allowing universal accessibility. All hardware modules including LEDs, RFID reader, and OLED display must interface smoothly with the ESP32 using standard protocols such as GPIO and I^2C, ensuring compatibility and ease of expansion.

4.2.10 PORTABILITY AND DEPLOYMENT FLEXIBILITY

The hardware arrangement is compact and lightweight, making it easy to transport between labs, classrooms, or demonstration venues. Installation requires only simple wiring, enabling quick setup without the need for professional tools. The modular nature of the prototype allows it to be dismantled and reassembled efficiently, supporting flexible deployment in various learning or testing environments.

4.3 SOFTWARE REQUIREMENTS

4.3.1 ARDUINO IDE (FIRMWARE DEVELOPMENT PLATFORM)

The Arduino IDE serves as the primary environment for developing and uploading firmware to the ESP32 microcontroller. It provides a clean and intuitive workspace with built-in tools for code editing, compiling, and debugging. The serial monitor assists developers in observing live system behaviour, tracking variables, and diagnosing issues related to timing, networking, or peripheral communication. Because the IDE is compatible with Windows, macOS, and Linux, it allows smooth deployment across multiple systems and diagnosing issues related to timing, networking, or peripheral communication without requiring extensive setup, making it highly suitable for rapid prototyping and iterative firmware development.

4.3.2 ESP32 BOARD SUPPORT PACKAGE (BSP)

To enable ESP32 development within the Arduino IDE, the official Board Support Package must be installed. The ensures seamless communication between the IDE and the microcontroller This package provides the essential compiler toolchain, low-level drivers, and hardware-specific libraries required for interfacing with the ESP32's Wi-Fi radio, GPIO pins, timers, and networking features. Because the IDE The BSP ensures seamless communication between the IDE and the microcontroller, allowing firmware uploads to occur reliably and guaranteeing that all ESP32 functionalities used in the traffic management system perform correctly.

4.3.3 PROGRAMMING LANGUAGE: EMBEDDED C/C++

The internal logic of the traffic system is implemented using Embedded C/C++, a language that offers direct access to hardware resources and efficient execution on microcontrollers. Through this language, the ESP32 manages GPIO operations for LED control, processes network tasks, updates the OLED display, and communicates with the RFID module. Its lightweight nature makes it ideal for IoT environments where memory and processing power must be used efficiently. Embedded C/C++ also ensures reliable timing, predictable behaviour, and long-term operational stability.

4.3.4 WEB DASHBOARD TECHNOLOGIES (HTML, CSS, JAVASCRIPT)

The interactive dashboard hosted on the ESP32 is created using widely supported web technologies. HTML defines the structure of the interface, while CSS styles the layout to ensure clarity and responsiveness on different screen sizes, including laptops, tablets, and smartphones. JavaScript handles real-time updates and enables asynchronous communication with the ESP32, allowing users to modify timings, trigger emergency mode, and monitor live status

without refreshing the page. Since the ESP32 serves the dashboard internally, it does not require external hosting or additional applications to function.

4.3.5 ESP32 WI-FI AND WEBSERVER LIBRARIES

The ESP32's Wi-Fi libraries provide the networking capability required for hosting the dashboard and exchanging data with the user. These libraries support both Access Point and Station modes, enabling deployment in standalone or network-connected environments. The webserver libraries manage routing for various HTTP endpoints, process incoming requests, and deliver JSON responses that represent real-time system status. Through these libraries, the controller updates traffic signals instantly in response to user commands while maintaining a stable communication link.

4.3.6 OLED DISPLAY LIBRARIES (ADAFRUIT GFX AND SSD1306)

The OLED module is managed using a combination of the Adafruit GFX library and the SSD1306 driver. These libraries simplify the process of rendering text, updating visual content, and refreshing the screen. They enable the display of active direction, signal colour, countdown values, and emergency alerts without interrupting the main traffic-control logic. Their efficient, non-blocking design ensures that display updates do not interfere with timing operations or webserver communication, making them ideal for real-time monitoring.

4.3.7 RFID LIBRARY (MFRC522)

When RFID authentication is used for emergency-vehicle detection, the MFRC522 library handles the communication between the ESP32 and the RFID reader. It manages low-level SPI communication, scans RFID tags, reads their unique identifiers, and verifies whether they match authorized emergency codes. Upon detecting a valid tag, the library helps the firmware execute the emergency-priority routine immediately. The MFRC522 library is optimized for quick scanning and stable operation, making it well suited for real-time traffic control applications.

4.3.8 DEVELOPMENT OPERATING SYSTEM

The project can be developed on any major operating system, including Windows, Linux, or macOS. The chosen system hosts the Arduino IDE, manages the USB communication interface used to program the ESP32, and stores all required libraries and source files. The operating system also supports compilation, firmware uploading, serial debugging, and file management, enabling a smooth development workflow without the need for specialized software environments.

4.3.9 WEB BROWSER FOR DASHBOARD INTERACTION

The system is designed to work with any modern web browser such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari. Through the browser, users can access the ESP32's dashboard over Wi-Fi, adjust signal timings, activate or deactivate emergency mode, reset the system, and observe real-time status updates. Because the interface runs entirely within the browser, it does not require installation of any additional applications, ensuring seamless access from smartphones, tablets, or laptops.

4.4 HARDWARE REQUIREMENTS

4.4.1 ESP32 MICROCONTROLLER

The ESP32 functions as the core processing unit of the entire traffic management system. It executes the programmed traffic-light sequence, handles real-time Wi-Fi communication, hosts the browser-based dashboard, and responds instantly to emergency inputs. With its dual-core architecture and integrated wireless capability, the ESP32 efficiently manages multiple parallel tasks such as LED control, OLED display updates, timing calculations, and HTTP request handling. Its low-power design and abundance of GPIO pins make it highly suitable for continuous IoT-based automation and long-duration operation.

4.4.2 RED–YELLOW–GREEN TRAFFIC LEDS

Each direction North, East, and South is equipped with a set of Red, Yellow, and Green LEDs to accurately represent a standard traffic signal. These LEDs operate on minimal power and can be driven directly by the ESP32's GPIO pins without the need for complex circuitry. Their quick and precise switching behaviour allows the system to mimic real-world traffic light transitions, making them ideal for demonstration and functional testing of the signal cycle.

4.4.3 BLUE EMERGENCY INDICATOR LED

The blue LED serves as a dedicated indicator for emergency-priority mode. When an emergency command is issued either through the dashboard or via RFID authentication, this LED turns on immediately, signalling that normal traffic sequencing has been overridden. This visual cue provides clear confirmation that an emergency vehicle is being prioritized and ensures observers can easily identify when the system is in override mode.

4.4.4 OLED DISPLAY

The SSD1306 OLED display provides an essential on-device interface for monitoring real-time system behaviour. It continuously shows the currently active direction, the present signal colour, the remaining time for the ongoing phase, and any emergency notifications. The display's high contrast and compact size make it well suited for quick on-site monitoring, allowing users to understand the system status instantly without needing to access the dashboard.

4.4.5 RFID READER

An RFID reader module, such as the RC522, can be integrated to detect authorized emergency vehicle tags. When a valid tag is scanned within range, the reader communicates with the ESP32 to activate emergency-priority mode automatically. This process eliminates the need for manual intervention, enabling fast, automated lane clearance for emergency responders. Its reliability and ease of interfacing using SPI communication make it highly effective for prototype-level smart traffic applications.

4.4.6 POWER SUPPLY AND VOLTAGE REGULATION

The system operates on a stable 5V USB power source, which provides consistent energy to the ESP32 and connected peripherals. Current-limiting resistors protect the LEDs from excessive current, while proper voltage regulation ensures the OLED and controller maintain uninterrupted performance. Effective power management is crucial for preventing flickering, overheating, or unexpected resets during prolonged operation, especially in demonstration settings.

4.4.7 SUPPORTING HARDWARE (RESISTORS, JUMPER WIRES)

Supporting components such as resistors, jumper wires, and a breadboard or printed circuit board are used to secure and organize the electrical connections. These elements ensure neat wiring, stable circuit behaviour, and easy reconfiguration during development or troubleshooting. Their modular nature allows the system to be assembled, tested, and modified without difficulty, making them essential for prototyping. These elements ensure neat wiring, stable circuit behaviours.

4.4.8 USB CABLE AND ESSENTIAL TOOLS

A standard USB cable is utilized both for powering the ESP32 and for uploading the firmware during development. overall circuit integrity. These tools play an important role in maintaining a stable and durable hardware setup suitable for continuous operation. tools including a soldering iron, multimeter, and connector pins assist in assembling reliable connections, identifying wiring issues, and ensuring.

4.5 OTHER REQUIREMENTS

4.5.1 STABLE WI-FI OR ACCESS POINT CONDITIONS

For the ESP32 to effectively host the dashboard and process user commands, a reliable wireless environment is required. Whether the board is operating in Station mode or generating its own Access Point, the surrounding network should have minimal interference to ensure quick communication and smooth interaction.

4.5.2 PROPER PHYSICAL SETUP AND SAFE WIRING PRACTICES

A tidy and well-organized circuit layout is necessary for accurate demonstration of the traffic system. All LEDs must be connected with appropriate resistors to prevent overheating, and wiring should be arranged securely to avoid loose connections or accidental short circuits especially during movement or transport of the prototype.

4.5.3 STABLE AND SUFFICIENT POWER SOURCE

To ensure continuous operation, the ESP32 and peripheral modules require a consistent 5V supply. While USB power works for general testing, longer demonstrations may require a regulated adapter or power bank to prevent unexpected resets or fluctuations in LED brightness.

4.5.4 SUITABLE LIGHTING ENVIRONMENT FOR DEMONSTRATIONS

Moderate indoor lighting is recommended to maintain clear visibility of the LEDs and OLED display. Very bright environments may wash out the display, while dark conditions may distort LED perception. Balanced lighting helps observers clearly view signal transitions and emergency indicators.

4.5.5 BASIC FAMILIARITY WITH SYSTEM OPERATION

Users should understand how to access the web dashboard, adjust timing parameters, enable emergency mode, and interpret status messages on both the dashboard and OLED. Basic operational knowledge prevents misuse, ensures proper configuration, and supports smooth demonstration flow.

4.5.6 AVAILABILITY OF PROJECT DOCUMENTATION AND CONFIGURATION FILES

Essential documentation such as wiring diagrams, firmware backups, dashboard source files, and configuration settings must be maintained and easily accessible. These records enable quick system restoration, updates, or troubleshooting in case of unforeseen errors or modifications.

CHAPTER 5

PROJECT DESIGN

5.1 SYSTEM DESIGN

The system design of the IoT-Based Emergency Priority Traffic System brings together hardware components, embedded firmware, wireless communication, and a browser-based interface into a unified operational framework. The intention behind this architecture is to create a compact yet realistic traffic-junction prototype capable of demonstrating automated signal sequencing, remote configuration through IoT connectivity, and immediate priority handling for emergency vehicles. At the core of the system lies the ESP32 microcontroller, which

functions as the main decision-making unit and coordinates every subsystem involved in traffic management.

The ESP32 executes all essential computational tasks, including the generation of timing routines, control of sequential Red–Yellow–Green signal changes, handling of emergency interruptions, and continuous updates to the OLED display. Its built-in Wi-Fi module removes the need for any external networking hardware, enabling the entire setup to operate independently as a self-contained IoT device. This not only simplifies the overall design but also ensures low latency, fast response, and stable performance even during prolonged operation.

From a structural perspective, the design is divided into several logical layers to maintain clarity, modularity, and scalability. The hardware layer includes components such as the RYG LEDs for each traffic direction, the blue emergency-indicator LED, resistors, jumper wires, the RFID reader (where implemented), and the OLED display. These elements are connected directly to the ESP32 through its GPIO pins and arranged to resemble a scaled-down three-way road junction. This layout allows observers to visually track how the signal transitions occur and how emergency overrides affect the cycle.

Above this lies the control-logic layer, where the system's core functionality is implemented in Embedded C/C++. This layer governs phase transitions, ensures that only safe and valid combinations of lights are displayed, and prevents two directions from receiving Green signals simultaneously. The emergency-priority logic temporarily suspends the automatic sequence whenever a dashboard command or RFID tag triggers an override, and safeguards are integrated to ensure smooth recovery when the system returns to normal operation. The communication layer relies on the ESP32's Wi-Fi capabilities to establish real-time interaction between the microcontroller and external users Depending on the configuration, the ESP32 can operate either as a standalone access point or as a station within an existing network. Commands such as timing updates, emergency activations, and reset requests are transmitted via lightweight HTTP communication, allowing the dashboard to function seamlessly across all devices without requiring additional software.

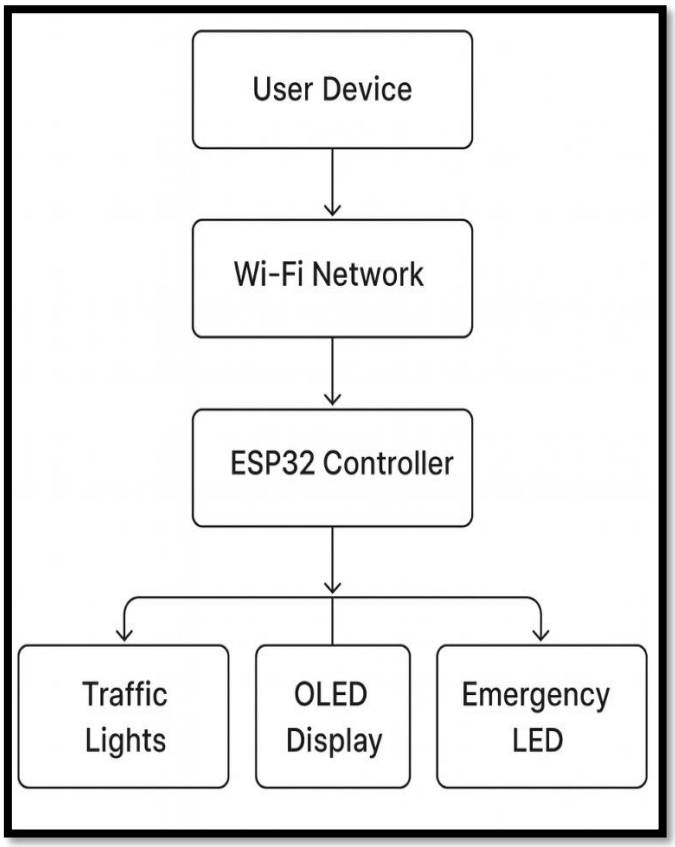

Fig 5.1 System Design Diagram

In Figure 5.1, the diagram shows Wi-Fi–based control between a user device and an ESP32 unit. The controller drives traffic signals, an OLED display, and an emergency indicator.

5.1.1 USER INTERFACE DESIGN

The User Interface (UI) of the IoT-Based Emergency Priority Traffic System is developed as a web-based dashboard hosted directly on the ESP32 microcontroller. Because the interface is accessed through any Wi-Fi enabled device such as a smartphone, laptop, or tablet users do not require external software installations or special applications. This approach ensures high accessibility, ease of use, and smooth remote operation for both technical and non-technical individuals.

One of the central components of the interface is the Traffic Signal Visualization Panel. This section provides a graphical representation of the three controlled directions North, East, and South along with the current Red, Yellow, or Green signal assigned to each one. The panel updates continuously, reflecting real-time LED transitions and clearly indicating which direction is active. It also displays a countdown timer for the current phase and changes its appearance during emergency mode activation. Through this visual layout, users can understand the ongoing traffic sequence at a glance and follow the flow of the system without observing the hardware directly.

The Control and Configuration Panel forms the interactive core of the dashboard. It allows users to modify operational settings such as the Green, Yellow, and Red durations for each direction. These adjustments are applied immediately to the running system, without requiring any restart. The interface also includes an option to apply a uniform set of timings to all directions simultaneously, streamlining configuration during testing or demonstrations. Additionally, dedicated emergency-activation buttons are provided for each lane, enabling instant priority clearance whenever required. A reset control is available for returning the system to its normal automatic cycle once the emergency has passed.

To support continuous monitoring, the dashboard incorporates a Status and Feedback Section. This area displays real-time information such as the current operating mode, the direction that holds the active signal, and the remaining time in the ongoing phase. The dashboard uses lightweight JavaScript polling at regular intervals to refresh the displayed values, ensuring that users receive up-to-date system feedback without noticeable delays. Together, these elements make the interface highly responsive, informative, and effective for both demonstration and operational purposes.

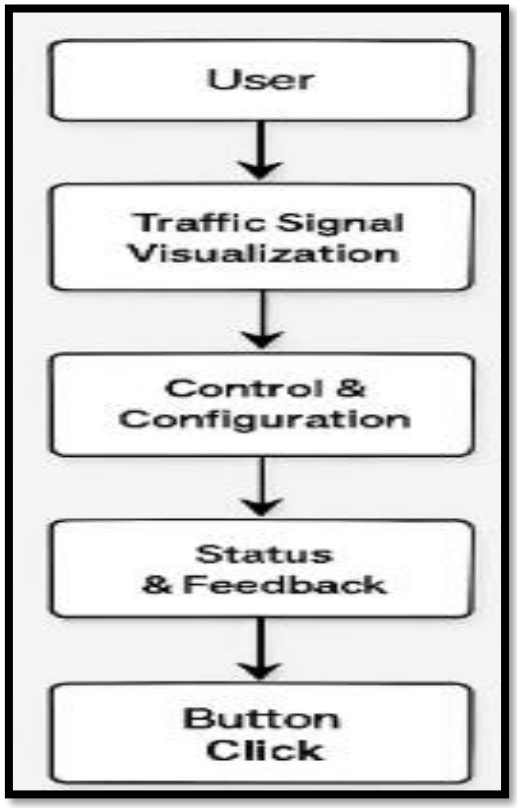

Fig 5.2 UI Flow Diagram

In Figure 5.2, the flow illustrates user interaction with the traffic system, starting from signal visualization to control configuration. User actions generate system status updates and feedback through interface commands.

5.1.2 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) illustrates the movement of information within IOT-Based Emergency Priority Traffic System and highlights how each subsystem interacts with the ESP32 microcontroller. Because the ESP32 serves as the central processing and communication hub, data exchange across the system remains streamlined, predictable, and efficient.

The data flow begins at the user interface, which is delivered through the web dashboard hosted directly on the ESP32. Once the controller is powered, it creates or joins a Wi-Fi network, allowing any compatible device such as a laptop, smartphone, or tablet to access the dashboard. From this interface, users can issue a variety of commands including adjusting signal timings, modifying traffic density settings, initiating emergency priority mode, or restoring normal automation. Each action is transmitted to the ESP32 using lightweight HTTP request messages.

Inside the microcontroller, these requests are interpreted by the internal firmware. The ESP32 updates the required timing parameters, alters the traffic phase, activates emergency logic, or pauses the standard sequence depending on the user command. Based on these decisions, the controller sends corresponding output signals to the hardware components. For example, the traffic LED assemblies are updated according to the current state, enabling each direction to display the appropriate red, yellow, or green indication.

When emergency mode is triggered, the ESP32 also updates the blue emergency indicator LED, providing a clear visual cue that normal sequencing has been overridden to prioritize an emergency route.

If the system includes an RFID reader, it functions as an additional data source. When an authorized emergency vehicle tag enters detection range, the reader forwards its unique ID to the ESP32. The controller verifies the tag and, if valid, activates the emergency-priority routine. Once the emergency condition ends, the firmware automatically returns to the normal signal cycle. Behind the scenes, lightweight JavaScript polling updates these values every half-second without overloading the network. This continuous feedback provides clarity, prevents user confusion, and supports decision-making during live demonstrations.

Simultaneously, the ESP32 continuously sends operational feedback to the OLED display, which acts as a local status interface. The is the display presents key information such as the active direction, the current light phase, remaining countdown, and emergency alerts. At the same time, the controller needs to be delivers structured JSON responses back to the web dashboard, ensuring that remote users receive real-time system updates without noticeable delay.

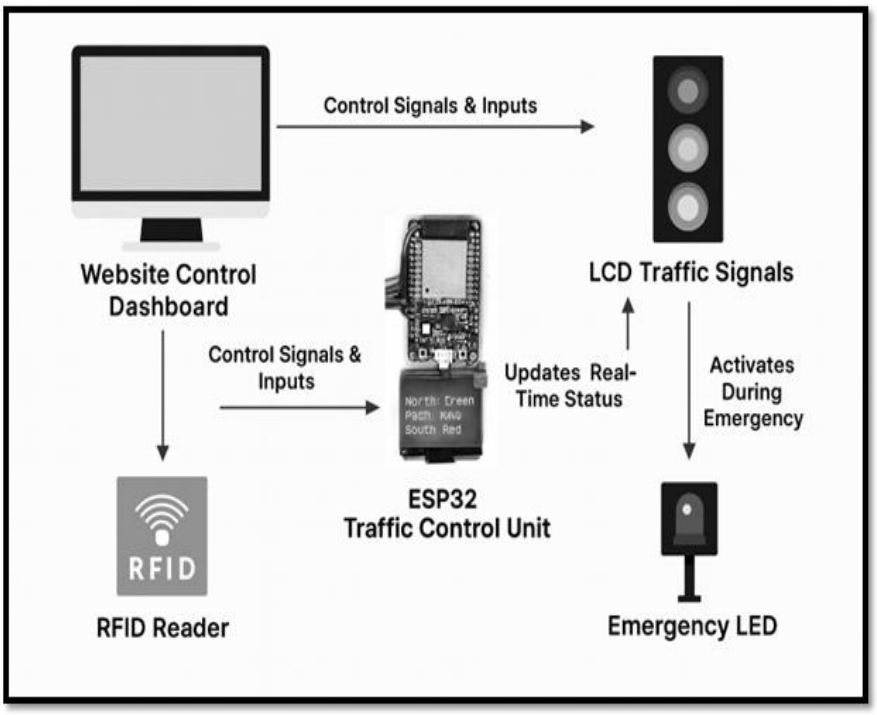

Fig 5.3 Data Flow Diagram

In Figure 5.3, the architecture presents an ESP32-based traffic control unit interfaced with a web dashboard and RFID reader for command input. The controller updates traffic signals in real time and triggers an emergency LED during priority conditions.

5.1.3 UML DIAGRAM

5.1.3.1 USE CASE DIAGRAM

The first use case concerns viewing the current traffic-signal status. Through the web dashboard, users can continuously monitor which direction—North, East, or South—holds the active Green, Yellow, or Red indication. The dashboard retrieves live system data from the controller at regular short intervals via the /status endpoint, ensuring that the displayed information reflects the most recent internal state and allowing observers to follow phase changes almost in real time.

Adjusting signal timings constitutes a second key use case. Operators may modify the durations assigned to the Green, Yellow, and Red phases for each approach through the dashboard interface. When new timing values are submitted, the ESP32 processes and applies them immediately so that they take effect in the next active cycle; this approach removes the need for restarting the controller and enables rapid adaptation to changing traffic conditions.

The activation of emergency priority mode is another central capability. The dashboard provides dedicated controls to trigger instant priority clearance for a chosen direction. Upon

activation, the selected lane is switched to Green without delay while all other approaches are forced to Red, and the blue emergency indicator LED illuminates as a visible confirmation.

A related use case covers system reset functionality. The reset control returns the controller to standard operation after any emergency or manual override. this function causes the system to exit emergency mode, resume the automatic sequencing routine, and restart the internal timers for the appropriate phase, thereby restoring a stable and predictable traffic cycle.

Automatic emergency activation via RFID represents an automated alternative to manual dashboard control. When an authorized RFID tag is detected by the reader, the ESP32 immediately transitions into emergency mode, activating the blue indicator and updating the OLED to communicate the override status. The system remains in this priority state until a reset command returns it to normal operation.

The operation of the physical traffic-LED outputs is a fundamental use case that follows from user or RFID actions. Every command or detection event results in a controlled LED response coordinated by the controller, ensuring correct sequencing from Green to Yellow to Red, safe transitions between directions, and prevention of any simultaneous conflicting signals.

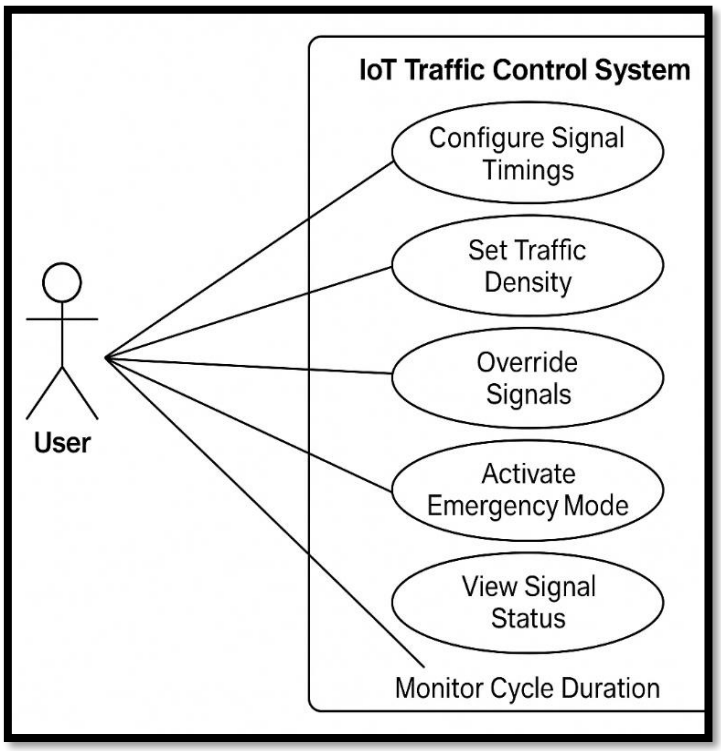

Fig 5.4 Use Case Diagram

In Figure 5.4, the use-case diagram illustrates user interaction with an IoT-based traffic control system. It highlights functions such as signal configuration, traffic monitoring, emergency activation, and real-time status observation.

5.1.3.2 ACTIVITY DIAGRAM

The activity of the IoT-Based Emergency Priority Traffic System begins with the system initialization sequence. When the ESP32 is powered on, it immediately runs a series of startup procedures to prepare all internal modules for operation. During this phase, the microcontroller configures all required GPIO pins for the Red, Yellow, and Green LEDs, initializes the OLED display, activates the Wi-Fi interface, and sets up the RFID reader if it is part of the configuration. The emergency indicator LED is also prepared at this stage. Once these peripherals are ready, the OLED presents a brief startup message confirming that the system has successfully completed its boot process.

The ESP32 attempts to establish a Wi-Fi connection. After joining the specified network, the microcontroller launches an embedded web server, making the traffic-control dashboard accessible to any user connected to the same network through a web browser. At this point, remote users can begin monitoring the system or adjusting its parameters through a mobile phone, tablet, or computer. With the communication layer active, the controller transitions into its main operational routine.

The next activity involves initiating the automatic traffic-light cycle. The ESP32 continuously rotates between the North, East, and South directions, assigning each lane a sequence of Green, Yellow, and Red phases. This sequence is governed by a non-blocking timing mechanism based on the millis() function, allowing the system to transition smoothly between phases while still responding to external requests and maintaining background processes such as RFID scanning.

During normal operation, the controller persistently checks for dashboard commands. These may include timing adjustments, emergency activation requests, or reset inputs. Each request reaches the ESP32 as an HTTP message and is processed immediately to maintain system responsiveness. The system must also monitor the RFID module, which continuously scans for incoming tag data. When a valid emergency-vehicle tag is detected, the ESP32 immediately interrupts the normal cycle and transitions to emergency-priority mode.

In emergency mode, the priority direction is forced to Green, while all other directions switch to Red to guarantee a safe and unobstructed route for the emergency vehicle. The OLED display updates to show an “Emergency Active” message, while the blue indicator LED illuminates, providing visual confirmation of the override.

Throughout both normal and emergency operations, the ESP32 continuously updates all output components. The traffic LEDs are adjusted according to the current phase, the blue emergency LED reflects the system’s state, and the OLED shows the active direction, signal colour, countdown time, and emergency status. These updates occur constantly, ensuring that both local observers and remote users receive real-time information.

Once emergency mode is cleared, the controller seamlessly returns to the automatic cycle and resumes regular signal sequencing. Monitoring of dashboard inputs and RFID scans continues in the background, forming an ongoing loop that defines the continuous behaviour of the traffic-control system. This activity structure ensures uninterrupted, safe, and adaptive traffic management suited for real-world demonstration.

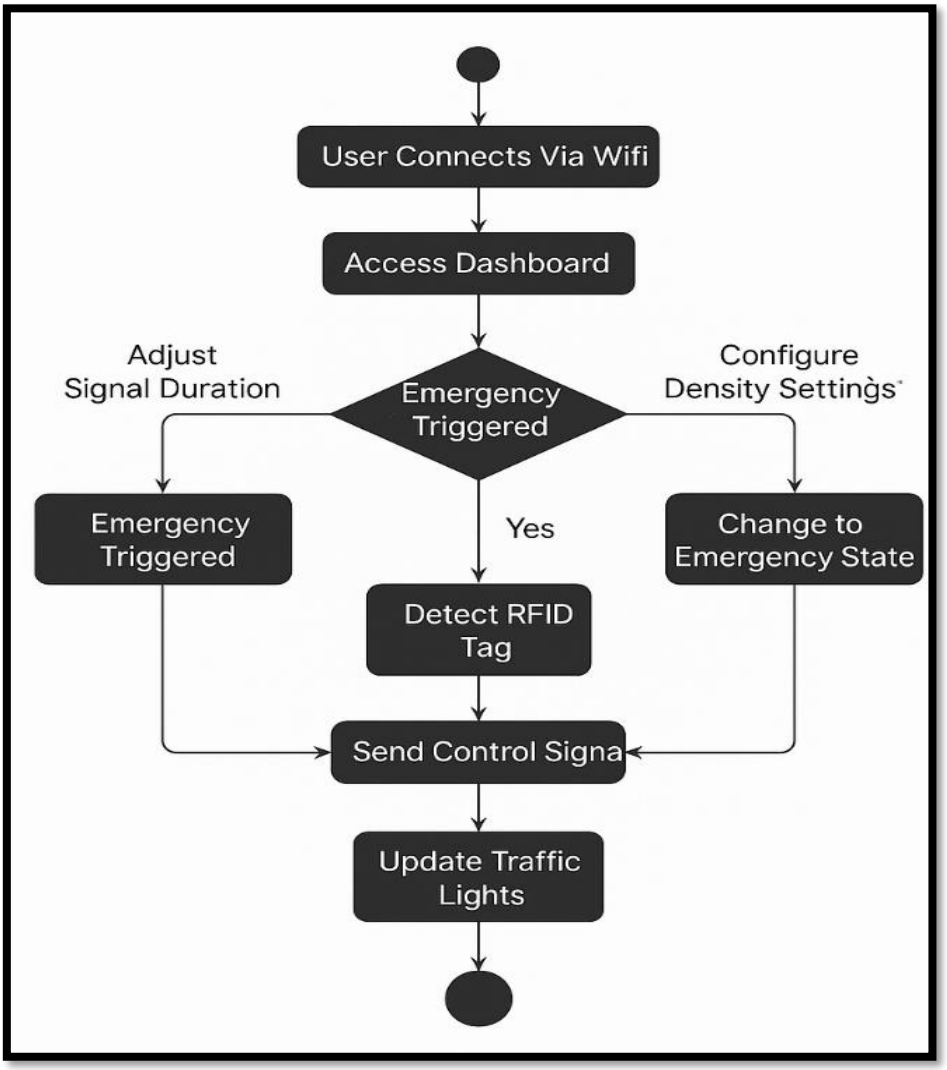

Fig 5.5 Activity Diagram

In Figure 5.5, the flowchart outlines the operational sequence of a Wi-Fi–enabled traffic control system, beginning with user access to the dashboard. It shows emergency detection through RFID and the subsequent control actions that update traffic signal states.

5.1.3.3 CLASS DIAGRAM

The ESP32Controller class represents the central logic engine of the system. It encapsulates the traffic sequencing algorithm, maintains the current active direction and phase state, and coordinates all peripheral modules. In addition to driving the LED outputs, this class interprets incoming HTTP commands from the dashboard, applies timing updates, executes

emergency-priority logic, and relays status information to the OLED display. It also interfaces with the RFID reader to receive tag data and decide whether to invoke the emergency override. As the primary orchestrator, the ESP32Controller is responsible for enforcing safety constraints and ensuring that only valid state transitions occur.

Each traffic approach (North, East, South) is modelled by an instance of the Traffic Light class. A Traffic Light encapsulates the pins and control routines required to switch its Red, Yellow, and Green LEDs. It provides simple methods to present a given state on the hardware, handle intermediate transitions safely, and report its current visual state back to the controller. By abstracting LED control into this class, the system keeps direction-specific logic modular and easy to extend for additional lanes.

The Web Dashboard class handles all browser-based interactions and acts as the user-facing interface to the ESP32Controller. This class serves the HTML/CSS/JavaScript payload, parses incoming HTTP requests, and translates them into controller actions such as timing updates, emergency triggers, or reset commands.

It also formats and returns JSON status responses for real-time polling, enabling the front-end to present an up-to-date view of the signal states. By separating presentation and network handling from core control logic, the design preserves clarity and simplifies maintenance.

On-device feedback is managed by the OLEDDisplay class, which encapsulates routines for rendering textual and symbolic information on the SSD1306 screen. The OLED Display receives structured status updates from the ESP32Controller and renders the active lane, current colour, remaining countdown, and emergency indicators in a readable format. Its non-blocking update methods are designed to coexist with the controller's timing logic so that display refreshes never interrupt signal sequencing.

The RFID Module class abstracts the RFID reader functionality. It continuously scans for tag UIDs, validates incoming identifiers against an internal whitelist or authorization list, and notifies the ESP32Controller when a recognized emergency tag is detected. This separation of responsibilities ensures that tag-processing complexity remains contained and that the controller can focus on decision-making rather than low-level RFID mechanics.

The Emergency LED class implements the simple yet important behaviour of the blue emergency indicator. It provides methods to activate and deactivate the LED in response to the controller's emergency state transitions. Although functionally minimal, this class centralizes the emergency-indication logic and makes it straightforward to alter the physical cue (for example, to change brightness or blink patterns) without modifying the main controller code.

The RFID Module pushes events to the controller when tags are detected, while the Emergency LED simply responds to control signals from the ESP32Controller. The controller also issues one-way updates to the OLED Display, which passively renders the provided information. This class-level organization enforces modularity, eases testing, and supports future extensions such as additional sensors or multi-junction coordination.

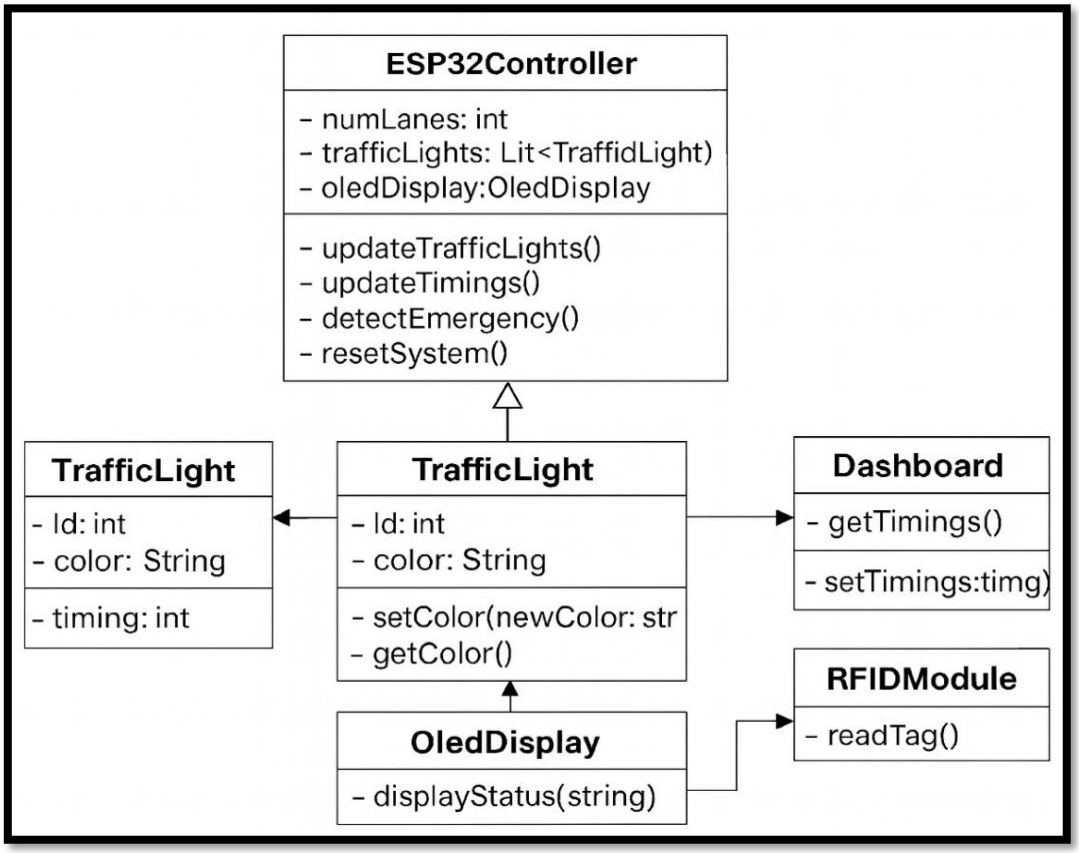

Fig 5.6 Class Diagram

In Figure 5.6, the class diagram represents the structural design of an ESP32-based traffic control system. It shows the interaction between the controller, traffic lights, dashboard, RFID module, and OLED display for coordinated traffic management.

5.1.3.4 SEQUENCE DIAGRAM

The sequence begins when a user opens the web dashboard in a browser, thereby establishing an initial communication channel with the ESP32 controller. Once the interface loads, the user may interact with the dashboard repeatedly: adjusting phase durations, initiating emergency priority, resetting the controller, or simply observing the live traffic status. Each user action on the front-end generates an HTTP request that is sent to the ESP32; these requests carry instructions such as timing updates, emergency activations, or reset commands.

When the controller receives a command, it parses the incoming request and updates its internal state variables accordingly. If the command requires a mode change for example, switching into emergency-priority the controller alters its operating state immediately and

recalculates the output logic. Following this decision stage, the ESP32 issues updated outputs to the connected hardware. The traffic LEDs are set to their new Red, Yellow, or Green states, the blue emergency indicator is activated if needed, and the OLED display is refreshed to show the active direction, current colour, remaining time, and any emergency notifications. These hardware updates provide immediate, visible confirmation of the controller's actions.

After the controller has processed the request and driven the hardware, it returns a structured JSON response to the dashboard. This response contains the current direction, signal phase, remaining countdown seconds, and emergency status indicators. Upon receiving the reply, the dashboard refreshes its user interface with the latest information, enabling the user to monitor the live system behaviour, verify countdown timers, and confirm that the requested commands were carried out successfully. This closed-loop exchange user action, controller processing, hardware update, controller feedback, and dashboard refresh repeats continuously during operation, forming the core runtime interaction pattern of the traffic management system.

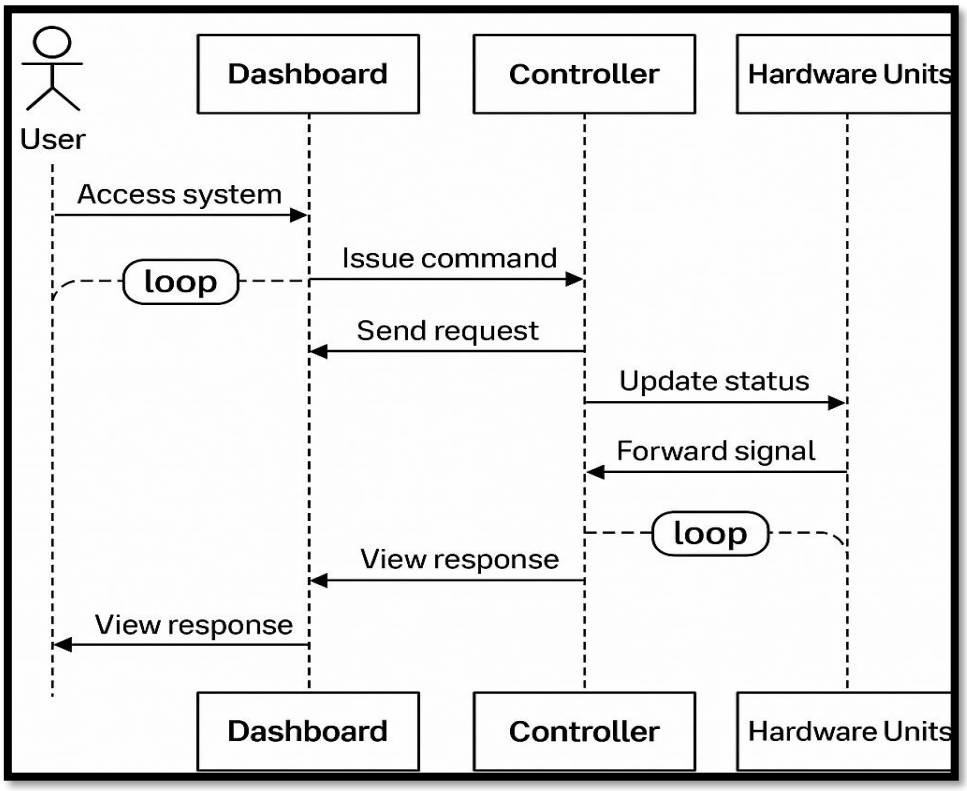

Fig 5.7 Sequence Diagram

In Figure 5.7, the sequence diagram illustrates the interaction flow between the user, dashboard, controller, and hardware units. It shows how commands are issued, processed, and reflected as system responses in a continuous control loop.

5.2 DETAILED DESIGN

5.2.1 MODULE-1: ESP32 TRAFFIC SIGNAL CONTROL MODULE

The Traffic Signal Control Module forms the core operational unit of the entire system and is implemented completely on the ESP32 microcontroller. This module is responsible for managing the full traffic-signal routine, coordinating phase transitions, executing emergency overrides, and maintaining communication with other modules such as the RFID reader, OLED display, emergency LED indicator, and the browser-based dashboard. Acting as the central brain of the system, the ESP32 ensures that all behaviours from automatic cycling to emergency intervention occur in a predictable, safe, and time-accurate manner.

At the heart of the design lies a software-based state machine that continuously cycles through the required traffic phases for each direction. The ESP32 processes the sequence **Green → Yellow → Red** for the North, East, and South lanes, ensuring that only one direction is allowed to display Green or Yellow at any given time. This prevents conflicting vehicle movements and reproduces the logic found in real-world signal controllers. The timing for each phase is stored in dedicated variables, measured in milliseconds, and the entire sequence is governed by the non-blocking millis() timer. This is that ensures that the system can multitask processing Wi-Fi requests or RFID to scans while still maintaining precise timing intervals and accuracy.

When the microcontroller powers on, it initializes all essential hardware components. This includes configuring GPIO pins for the red, yellow, and green LEDs, preparing the blue LED that indicates emergency priority, setting up the OLED display, and activating the RFID module when available. Acting as the central brain of the system, the ESP32 ensures that all behaviours from automatic cycling After initialization, the ESP32 loads default time parameters and begins the automatic signal cycle When no such events occur, the controller operates normally however, emergency events immediately interrupt the cycle, placing the system into priority mode.

The Emergency Override Handler is designed to take control whenever an emergency event is triggered through the dashboard or detected via RFID. Upon activation, the normal sequence is suspended, and the direction requiring priority is switched immediately to Green. All other directions are forced into Red, and the system remains in this locked-state configuration until the override is cleared. This component ensures that emergency vehicles can pass through the intersection without waiting for the normal cycle to complete.

The Output Driver manages all visual and external updates, synchronizing the ESP32's internal logic with hardware elements. This includes updating the red, yellow, and green LEDs, illuminating the emergency LED when needed, and refreshing the OLED display to show the current direction, active signal colour, remaining time, and emergency status. Acting as the

central brain of the system, the ESP32 ensures that all behaviours from automatic cycling Because these updates occur continuously and in real time, users monitoring the system either locally through the OLED or remotely via the dashboard always receive accurate, up-to-date information about the intersection's state.

5.2.2 MODULE-2: WEB DASHBOARD AND IOT COMMUNICATION MODULE

The Web Dashboard and IoT Communication Module provide the remote-interaction capability of the IOT-Based Emergency Priority Traffic System. It enables users to view real-time traffic status, adjust signal timings, activate emergency priority, and reset the system all through a standard web browser. This component transforms the ESP32 into a fully functional IoT device by combining Wi-Fi networking, HTTP communication, and embedded web-page hosting within a single microcontroller.

When the ESP32 begins operation, it configures its wireless mode based on the deployment setup. In Access Point (AP) mode, the ESP32 creates its own Wi-Fi hotspot, allowing users to connect directly without any external router. In Station (STA) mode, the microcontroller joins an existing Wi-Fi network and becomes accessible to all devices connected to the same router. After establishing the network link, the ESP32 launches an onboard HTTP server capable of delivering the dashboard interface and processing user commands.

The dashboard hosted by the microcontroller provides a graphical environment that displays the current traffic states for the North, East, and South directions. It also contains interactive elements such as timing configuration fields, emergency-activation buttons, and reset controls. These components enable complete remote supervision of the traffic system without requiring specialized software installations.

Internally, the ESP32 defines a series of HTTP endpoints that support communication between the dashboard and the controller. The root endpoint serves the dashboard interface, while the /status route supplies live system information in JSON format, allowing the webpage to refresh automatically. Other endpoints handle timing updates, emergency-direction commands, and system resets. Whenever the user interacts with the interface, the browser sends a corresponding HTTP request to the microcontroller, which interprets the parameters, updates internal variables, and responds with an acknowledgment or updated status data.

Once the ESP32 web server is initialized, the user's browser retrieves the dashboard and displays the interface. From that point onward, the dashboard becomes highly interactive through continuous background communication. JavaScript embedded within the webpage regularly sends requests to the /status endpoint, typically several times per second. These components enable complete to fetch live information such as the active direction, current

signal colour, countdown duration, and emergency-priority status. This ensures that the displayed data always matches the ESP32's internal state without requiring the user to refresh the page manually.

When a user modifies timing parameters or activates emergency priority for a specific lane, the dashboard sends an HTTP request with the updated information. The ESP32 processes the request immediately by adjusting its internal timing values or switching into emergency mode. It then returns a status response that confirms successful execution. These updates are seamlessly integrated with the Traffic Signal Control Module, which instantly applies the new values to the LED indicators, updates the OLED display, and modifies the system's operational state accordingly.

Through this design, the Web Dashboard and IoT Communication Module ensures smooth, low-latency interaction between the user and the traffic controller. Its ability to combine real-time monitoring with immediate command execution makes the entire system highly dynamic, intuitive, and effective for both demonstration and practical use

5.2.3 MODULE-3: EMERGENCY VEHICLE PRIORITY & RFID TRIGGER MODULE

The Emergency Vehicle Priority and RFID Trigger Module is responsible for ensuring that emergency responders such as ambulances, police units, and fire engines receive immediate and uninterrupted passage through the junction. This feature elevates the system beyond traditional signal controllers by replacing manual override procedures with a fully automated mechanism. Its integration within the IoT-based framework allows the controller to react instantly to emergency events while maintaining predictable and safe traffic behaviour. The module is activated either through the web dashboard or by detecting an authorized RFID tag, making it suitable for both manual supervision and automated field operation.

The module operates through two complementary activation pathways: dashboard-based emergency commands and RFID-triggered detection. When the system starts, the ESP32 configures and initializes the MFRC522 RFID reader via the SPI communication interface. Emergency vehicles carrying pre-registered RFID tags are automatically detected when they move within the reader's range. The RFID reader captures the unique identification number embedded in the tag and forwards it to the ESP32 for verification. If the ID matches the stored emergency credentials, the microcontroller shifts immediately into Emergency Mode. This transition overrides the ongoing Red–Yellow–Green sequence and assigns a Green signal to the lane that requires priority.

During emergency priority, the controller enforces strict rules to ensure safety and predictability. The active direction remains Green while all other directions are forced to Red,

thereby preventing conflicting movements at the intersection. A blue indicator LED illuminates to show that the system is in an emergency state. This visual signal is especially useful during real-world deployment or prototype demonstration, as it confirms to users that normal sequencing has been temporarily suspended. The OLED display mirrors this condition by showing a clear emergency alert message, ensuring that observers receive immediate on-device feedback.

The internal emergency-handling logic is structured to provide rapid and deterministic response. Whenever the RFID reader scans a tag, the ESP32 validates the received identifier against its list of authorized emergency tags. Upon identifying a valid match, the controller activates the emergency flag and assigns priority to the appropriate lane. The direction associated with the emergency vehicle immediately switches to Green, while all remaining approaches revert to Red. This guarantees an unobstructed route for the vehicle while ensuring that traffic from other lanes remains safely halted.

Throughout this mode, the controller suspends all automatic cycling routines. The system maintains emergency priority until it is explicitly cleared through the dashboard's reset function. Once the reset command is issued, the emergency indicator is turned off and the system resumes its normal operation from the next scheduled phase. This smooth transition ensures that the system remains both safe and efficient, returning to regular traffic management without timing errors or abrupt phase changes. The combination of RFID automation and dashboard control makes this module a highly reliable solution for emergency vehicle prioritization, significantly reducing response time and enhancing overall traffic safety.

5.2.4 MODULE-4: OLED STATUS DISPLAY MODULE

The OLED Status Display Module plays an essential role in delivering real-time, device-level visibility of the system's current operating conditions. Unlike the web dashboard, which requires a Wi-Fi-enabled device for monitoring, the OLED provides an immediate, on-hardware view of the traffic controller's behaviour. This makes it extremely valuable during demonstrations, laboratory use, troubleshooting, and scenarios where quick visual diagnostics are required. By presenting direction states, signal colours, countdown timers, and emergency alerts directly on the screen, the OLED ensures that the functioning of the entire system can be understood at a glance even without network connectivity.

The OLED module used in this project, typically an SSD1306-based display, communicates with the ESP32 microcontroller through the I2C protocol. This interface enables lightweight, fast, and reliable data transfer, allowing the display to refresh multiple times per second without affecting the controller's performance. Throughout system operation, the ESP32 continuously updates the screen with crucial information such as the currently active

traffic direction, the corresponding signal phase, and the remaining countdown time for that phase. During emergency conditions, the OLED switches to an alert-oriented layout, prominently showing emergency messages and highlighting the direction that has been granted priority. The module also displays initialization messages during system start-up, confirming that the hardware has been configured correctly and is ready for use.

The internal logic that governs the OLED output follows a well-structured routine to maintain clarity and responsiveness. Whenever a traffic phase changes either due to the normal signal cycle or a timing adjustment from the dashboard the ESP32 clears the screen and reconstructs the displayed content based on the updated values. It prints the signal status for each direction and applies emphasis to the currently active lane to improve readability. If emergency mode is active, the standard layout is replaced with an override screen that clearly communicates the emergency condition and the direction receiving Green priority. The display is refreshed continuously, ensuring that users always see the most recent state of the system.

This module greatly enhances system transparency by providing a reliable visual reference that mirrors the behaviour of both the LEDs and the dashboard interface. It helps confirm correct phase sequencing, supports real-time demonstrations, and offers immediate confirmation when emergency mode is triggered. Through its clear and consistent presentation of information, the OLED display significantly strengthens the usability and interpretability of the entire traffic-control prototype.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 MODULE-1: ESP32 TRAFFIC SIGNAL CONTROL MODULE

The Traffic Signal Control Module is responsible for managing the complete signalling workflow for the North, East, and South directions. Each direction contains a set of Red, Yellow, and Green LEDs, and the ESP32 cycles through these using a predefined sequence Green,

followed by Yellow, and then Red before shifting control to the next lane. This structured flow ensures smooth, conflict-free transitions. At the core of this module is an internal state machine that operates using two key variables: the currently active direction and the active phase, which may be Green, Yellow, or Red. This organized design keeps the system predictable, fast, and easy to manage.

To maintain responsiveness, the module uses a non-blocking timer approach based on the *millis()* function, avoiding delays that would freeze the microcontroller. This allows the ESP32 to update LEDs, communicate with the web dashboard, refresh the OLED display every half-second, and react instantly to emergency inputs. As a result, the system achieves efficient multitasking without interrupting ongoing operations.

Emergency vehicle handling is tightly integrated into this module. When an emergency request is triggered either through the dashboard or an RFID tag the selected direction immediately switches to Green, while all remaining directions are forced to Red. A blue LED activates to indicate emergency priority, and the OLED displays a message such as “Emergency Ambulance.” During this mode, the normal cycle pauses automatically and resumes only when the emergency is cleared.

The OLED display plays an important supporting role by presenting real-time system data, including the active direction, signal colour, countdown timer, and emergency notifications. This ensures clear on-site monitoring even without the dashboard.

This module works in close coordination with other subsystems: the Web Dashboard Module, which provides timing updates and emergency commands, and the RFID Module, which enables automatic detection of authorized emergency vehicles. Through synchronized interaction across all modules, the system maintains accuracy, stability, and safe signalling behaviour.

This module seamlessly integrates with other system components. It receives configuration and timing updates from the Web Dashboard Module, processes emergency signals from the RFID Module, and sends unified output to the LED system and OLED display. This tight synchronization between modules and enhances system accuracy, making the traffic management process both efficient and intelligent.

Module-1 ESP32 Traffic Signal Control Module

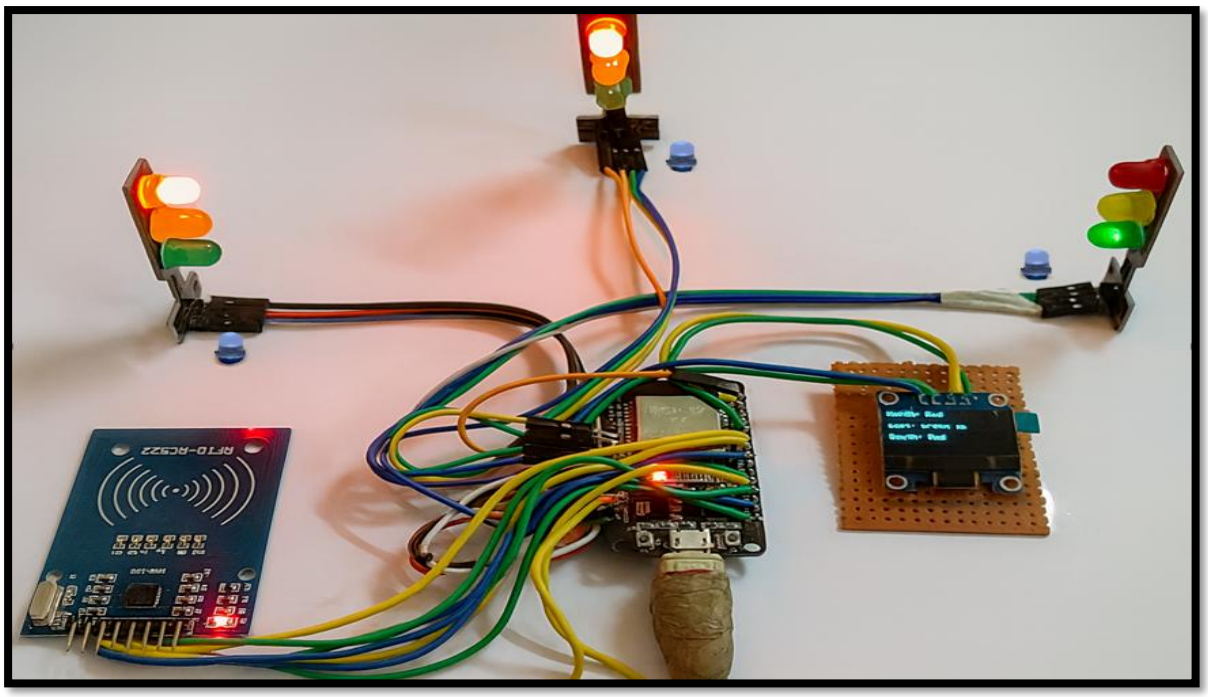

Fig 6.1 ESP 32 Connection Module

In Figure 6.1, The hardware prototype of the ESP32-based traffic control system is shown with connected traffic signal LEDs, an RFID module, and an OLED display. The setup demonstrates real-time signal operation and emergency detection through integrated components.

6.2 MODULE-2 WEB DASHBOARD & IOT COMMUNICATION

The Web Dashboard and IoT Communication Module enables users to remotely observe and control the traffic signal system through any standard web browser. This module operates on the ESP32's built-in Wi-Fi capability, which allows the microcontroller to function as a local web server. When powered on, the ESP32 generates an accessible IP address that users can enter into a mobile phone, laptop, or tablet to open the traffic control interface. This removes the need for specialized applications or external servers and ensures simple, universal access.

The dashboard itself is designed to provide a clean and intuitive interface for system interaction. Users can view the real-time signal status of all three directions North, East, and South along with visual indicators showing whether each direction is in Red, Yellow, or Green. The dashboard also includes options to modify signal timings individually or apply the same configurations to all directions simultaneously. In addition, dedicated emergency buttons allow the user to grant immediate priority to a selected direction, and a Reset button restores the system to automatic operation. These features make the interface highly interactive and flexible for both testing and demonstration scenarios.

Communication between the dashboard and ESP32 is achieved through lightweight HTTP requests. The ESP32 hosts several predefined routes such as / for loading the dashboard page, /status for returning JSON-formatted system data, /settimes for updating timing values, and /reset or /traffic1, /traffic2, /traffic3 for emergency control. Each user action—whether pressing a button or entering new timing values—triggers a specific HTTP request, which the ESP32 interprets and processes immediately. This direct client-controller communication ensures minimal delay and high responsiveness.

Integrates closely with the Traffic Signal Control Module (Module–1). When the user updates timings through the dashboard, Module–1 adjusts its internal timers accordingly. When emergency mode is triggered, the signal control logic is overridden instantly, and the chosen direction receives priority. Similarly, when reset commands are issued, Module–1 resumes the normal automatic cycle. This tight synchronization ensures that both modules operate harmoniously, resulting in a responsive, reliable, and intelligent IoT-based traffic management system.

Module-2 Web Dashboard & IOT Communication

Fig 6.2 Real-Time Traffic Control Dashboard Interface.

In Figure 6.2, the smart traffic control dashboard provides an interface for monitoring and configuring signal timings for multiple directions. It enables real-time adjustment of green, yellow, and red phases through a centralized control panel.

6.3 MODULE-3: RFID-BASED EMERGENCY VEHICLE PRIORITY SYSTEM

The Emergency Vehicle Priority Module ensures that ambulances and other emergency responders receive an immediate right-of-way at intersections. It operates through two methods: RFID-based automatic detection and manual activation from the web dashboard.

In the automated mode, each emergency vehicle carries an RFID tag. When the vehicle approaches the junction, the RFID reader detects the tag and sends its ID to the ESP32. If the ID matches a registered emergency tag, the controller instantly switches the system into Emergency Mode. This eliminates the need for manual intervention and provides fast, reliable priority clearance.

The dashboard also includes emergency buttons for North, East, and South directions, allowing operators to manually activate priority if needed. The interface includes dedicated emergency buttons for the North, East, and South directions. This feature is particularly useful for demonstration, testing, or situations where an emergency vehicle may not be equipped with RFID tags.

Once emergency mode is active, the system immediately sets the selected direction to GREEN and turns all other directions RED. The blue emergency LED lights up, and the OLED displays an alert message indicating that emergency clearance is active. During this period, the normal traffic cycle is paused to ensure a safe and unobstructed passage. When reset, the system returns seamlessly to automatic operation.

Module–3 RFID-Based Emergency Vehicle Priority System

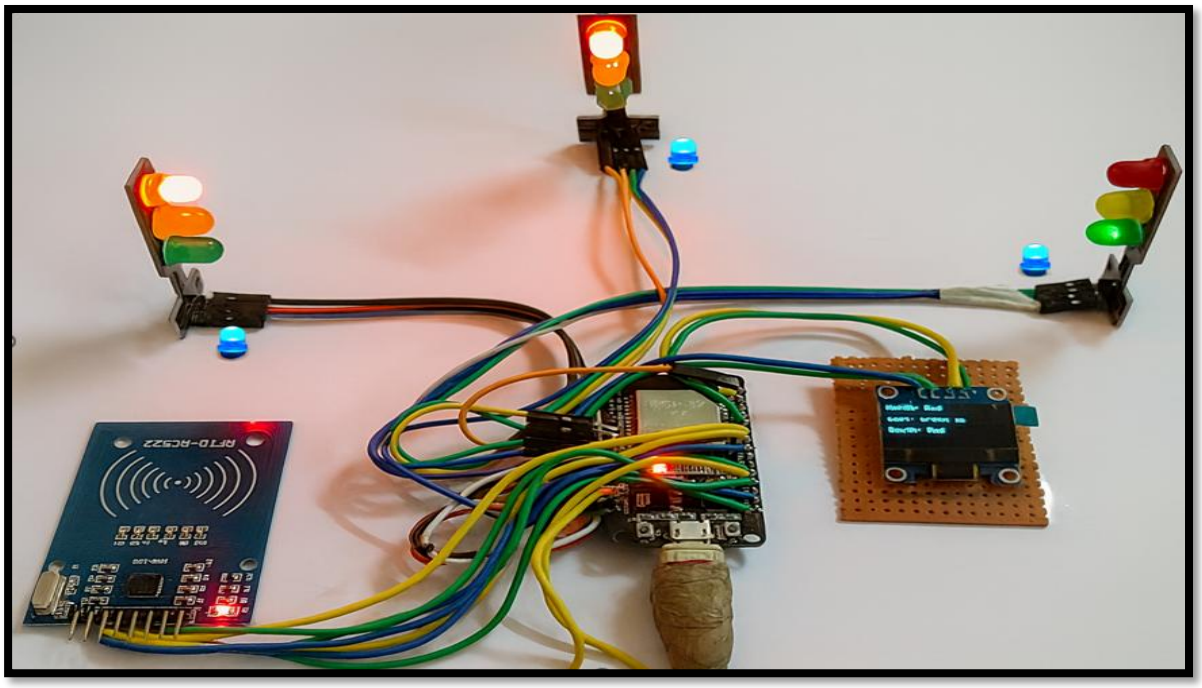

Fig 6.3 Emergency Vehicle In Line

In Figure 6.3, the experimental setup of the ESP32-based traffic control prototype is illustrated with connected signal LEDs, an RFID reader, and an OLED module. The configuration demonstrates integrated signal operation and emergency indication in real time.

6.4 MODULE–4: OLED STATUS DISPLAY MODULE

The OLED Display Module functions as the primary on-device interface for the Smart Traffic Management System. It provides continuous, real-time visual feedback, allowing users to understand the traffic signal status immediately without relying on the web dashboard. This makes the system especially useful during demonstrations, field testing, or situations where quick, local monitoring is required. By presenting essential information directly on the hardware, the OLED enhances the accessibility, clarity, and reliability of the system.

The main purpose of the OLED screen is to display the current operational state of the traffic controller. It shows the active direction (North, East, or South), the corresponding signal colour (Red, Yellow, or Green), and the remaining time for the ongoing phase. During system initialization, it displays startup messages that confirm correct booting and readiness. In emergency conditions, the OLED prominently displays alerts such as “Emergency Ambulance,” ensuring that the operator or bystander is instantly aware of priority mode activation. This real-time visibility greatly supports debugging, monitoring, and user interaction.

Internally, the OLED module (typically based on the SSD1306 driver) communicates with the ESP32 via the I2C protocol. Whenever the system logic updates whether due to a phase transition, a timing change from the dashboard, or an emergency override the ESP32 refreshes the display. The update process involves clearing the previous frame, printing lane names, marking the active direction, and showing countdown values if applicable. These updates occur rapidly and continuously, creating a smooth and responsive visual output without interfering with the controller’s timing functions.

The OLED mirrors the automatic signal cycle by showing each direction’s status in real time. For example, when East is in the Green phase with four seconds remaining, the OLED reflects this state exactly, ensuring perfect alignment between the hardware LEDs and the displayed information. This consistency reassures users that the internal logic and external indicators are fully synchronized.

In emergency mode, the OLED takes on an even more critical role. It immediately overrides the standard display to show emergency alerts and updated signal states, with the priority direction highlighted in Green and all others marked Red. This clear representation helps confirm that emergency handling has been activated correctly and safely.

Overall, the OLED Display Module significantly enhances the system's transparency and usability by providing an immediate, accurate, and easy-to-read account of the traffic controller's operations

Module–4 OLED Interface

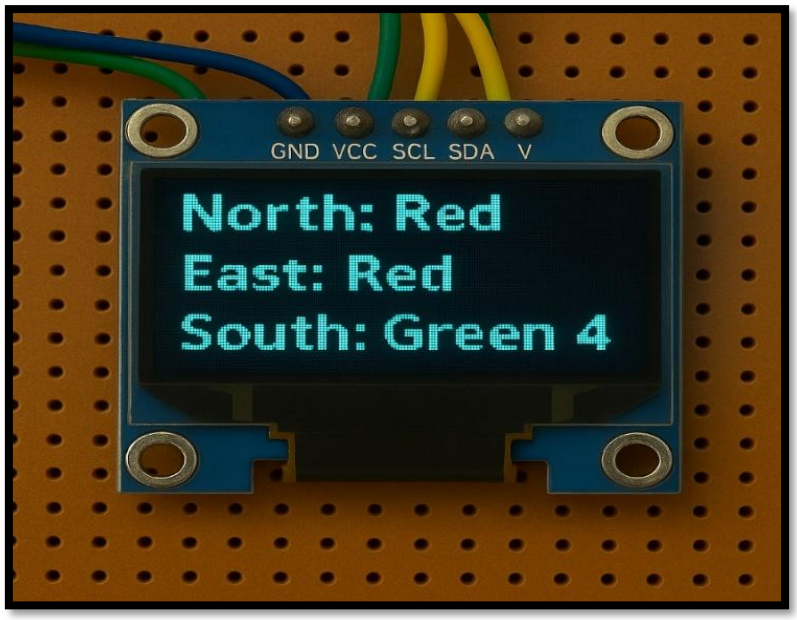

Fig 6.4 OLED Displaying Real-Time Traffic Status

In Figure 6.4, the OLED display presents the real-time status of traffic signals for multiple directions. It provides clear visual feedback on signal states and remaining phase duration.

‘

CHAPTER 7

TESTING

7.1 OBJECTIVES OF TESTING

The primary goal of the testing phase is to determine whether the IoT-Based Emergency Priority Traffic System performs reliably under various operating conditions and fulfils all of the intended functional requirements. During testing, special attention was given to the accuracy of the traffic-signal sequencing, ensuring that the ESP32 maintained the correct Green–Yellow–Red order for each direction without producing overlapping or conflicting outputs. This helped confirm that the fundamental control logic was stable and responding precisely to internal timing rules.

Another major focus was the evaluation of the emergency-priority feature. The system needed to demonstrate the ability to switch instantly into emergency mode whenever a command was received from the dashboard or when an authorized RFID tag was detected. Testing ensured that the selected lane shifted immediately to a Green signal while other lanes transitioned to Red without delay, and that the normal cycle paused safely until the emergency was cleared.

In addition to signal behaviour, the communication flow between the web dashboard and the ESP32 controller was examined. The system had to interpret timing changes, emergency triggers, and reset commands correctly, applying them in real time without noticeable latency. The OLED display was also monitored to verify that it consistently reflected the active direction, signal colour, remaining time, and emergency alerts, ensuring that on-device information stayed synchronized with dashboard and LED outputs.

A crucial part of testing involved the interaction among all modules controller, dashboard, RFID reader, LEDs, and OLED to ensure they operated smoothly as a single integrated system. Long-duration trials were conducted to observe whether the setup could run continuously without freezing, rebooting, or producing timing inconsistencies. Finally, the testing process also served to identify and correct any wiring issues, timing mismatches, logic flaws, or communication failures encountered during development. Through these evaluations, the system's reliability, usability, and overall operational stability were thoroughly validated.

The testing phase also provided valuable insights into the resilience and adaptability of the system. By simulating a variety of real-world traffic scenarios such as rapid changes in timing demands, repeated emergency activations, and fluctuating Wi-Fi connectivity the tests demonstrated how effectively the controller-maintained synchronization across all modules. Moreover, the testing process validated the robustness of the underlying logic and

communication layers, showing that the system can maintain stable performance without compromising safety or responsiveness.

7.2 TESTING METHODOLOGIES

7.2.1 UNIT TESTING

Unit testing formed the foundation of the evaluation process, where each component of the system was isolated and examined individually. During this stage, functions such as LED switching patterns, OLED initialization routines, Wi-Fi setup sequences, HTTP endpoint handling, and RFID tag reading operations were tested independently. This allowed the developers to verify that every module behaved as expected before being integrated into the larger system. By identifying issues early such as incorrect GPIO mapping, display rendering errors, or faulty tag detection the overall debugging effort was significantly reduced. This phase ensured that each part of the system was logically sound and technically stable before moving on to combined testing.

7.2.2 INTEGRATION TESTING

Once the individual units performed correctly, integration testing was carried out to observe how these modules worked together as a cohesive system. This phase focused on verifying that commands from the dashboard accurately updated the internal state of the ESP32 and that these changes were reflected consistently on the LED indicators and OLED display. Integration testing also confirmed that emergency requests from both the dashboard and RFID module triggered the appropriate responses without delay or conflict. The goal was to ensure seamless data flow between software and hardware, allowing all components to synchronize in real time. This step demonstrated that the system could handle concurrent operations without malfunctioning or producing inconsistent outputs.

7.2.3 SYSTEM TESTING

System testing involved evaluating the complete traffic management prototype under realistic operating scenarios. The traffic-light cycle was observed over extended periods to ensure that transitions between Green, Yellow, and Red states occurred at the correct intervals. Emergency-mode behaviour was tested repeatedly to verify that the system paused the normal sequence and resumed it correctly afterward. Adjustments to signal timings during active operation were also examined to confirm that the controller could adapt dynamically without restarting. This stage assessed the interaction between all system layers logic, communication, hardware, and user Conditions.

7.2.4 USER ACCEPTANCE TESTING (UAT)

User Acceptance Testing focused on how actual users experienced the system when interacting with the web dashboard. This included evaluating how easily users could navigate the interface, interpret the displayed information, and perform essential actions such as modifying timings or activating emergency mode. Feedback gathered during this stage was essential for identifying usability improvements, such as adjusting button placement or enhancing feedback messages. UAT demonstrated that the system was accessible even to individuals with no technical background, confirming that its design supports intuitive and efficient operation in practical scenarios.

7.2.5 STRESS AND RELIABILITY TESTING

Stress and reliability testing examined how the system performed when subjected to demanding conditions for extended periods. The ESP32 was allowed to run for long durations while experiencing rapid emergency toggles, repeated timing updates, and simulated Wi-Fi reconnection cycles. These tests helped reveal the system's resilience against heavy workloads and unpredictable inputs. The goal was to confirm that the controller did not freeze, reboot unexpectedly, or produce conflicting signal outputs under pressure. The system demonstrated stable behaviour throughout, proving that the underlying architecture was robust enough to handle continuous field operation.

7.2.6 HARDWARE TESTING

The final stage involved evaluating the physical hardware components to ensure consistent and reliable performance. LED assemblies were examined for brightness, switching accuracy, and response speed, while the OLED display was checked for readability under different lighting conditions. RFID detection was tested at multiple angles and distances to confirm tag recognition consistency. The wiring and power system were also inspected to ensure there were no loose connections, voltage drops, or overheating issues during prolonged operation. This testing ensured that the hardware was durable, stable, and capable of supporting real-time traffic control without failure.

The components maintained stable functionality even during repetitive testing cycles, indicating strong endurance for continuous use. The ESP32's GPIO outputs operated without signal degradation, and no noise-related interference affected LED or RFID performance. Temperature checks confirmed that the board and peripheral modules remained within safe operating limits. Overall, the hardware proved sufficiently robust to sustain long-term demonstrations and simulated real-world traffic conditions.

7.3 TEST CASES AND RESULTS:

Sl No	Test Case	Input Provided	Expected Output	Actual Output	Result
1	**Normal Signal Cycle**	System powered ON	LEDs cycle: Green → Yellow → Red for each direction	Cycle performed correctly	**Pass**
2	**Change Green Time (North)**	Green 0 = 8 seconds (from dashboard)	North Green stays ON for 8 seconds	North Green = 8 seconds	**Pass**
3	**Apply All Timings**	Green All=5, Yellow All=2, Red All=3	All directions update timings	All directions updated in real time	**Pass**
4	**Emergency Priority Enabled for One Lane**	Click “East Ambulance”	East → Green, others → Red, Blue LED ON	Exactly matched dashboard & hardware	**Pass**
5	**Reset to Auto Mode**	Click “Reset”	Emergency OFF, auto cycle resumes	Auto cycle resumed normally	**Pass**
6	**OLED Display Update**	System running	OLED shows active direction + timer	OLED updated correctly	**Pass**
7	**Wi-Fi Connectivity**	Connect via phone/laptop	Dashboard loads with live updates	Connected & updated every 0.5s	**Pass**
8	**RFID Emergency Trigger**	Tap emergency RFID tag	System enters emergency mode	Trigger successful	**Pass**
9	**Wrong Input Handling**	Negative timing value	System should reject input	Input ignored, safe behavior	**Pass**
10	**Long Duration Test**	Run for 1 hour	No freeze, no false LEDs	System stable for full duration	**Pass**

Table 7.1 Test Case and Results

7.4 PERFORMANCE EVALUATION

The performance of the IoT-Based Emergency Priority Traffic System was assessed after completing all functional, integration, and long-duration tests. Overall, the system demonstrated smooth, stable, and highly responsive behaviour under a variety of operating conditions. During evaluation, the ESP32 responded to user commands—such as timing adjustments, resets, and emergency triggers—within just a few hundred milliseconds. This fast reaction time ensured that changes in both the dashboard and the physical LED indicators appeared almost instantly. The OLED display also updated without delay, reflecting the current direction, signal colour, and remaining countdown in real time, which contributed to the system's overall responsiveness.

Timing accuracy was another strong aspect observed during testing. The programmed durations for the Green, Yellow, and Red phases remained precise throughout repeated cycles. Even during extended operation, the countdown shown on the OLED and dashboard stayed fully synchronized with the controller's internal timer, showing no noticeable drift. This consistency confirmed the reliability of the non-blocking timing approach used in the system.

The system also performed exceptionally well during continuous operation. A one-hour runtime test demonstrated that the controller could maintain automatic cycling without freezing, rebooting, or displaying conflicting LED states. Wi-Fi communication stayed steady, allowing the dashboard to refresh at regular intervals without interruption. The ability to run for long periods without instability highlighted the robustness of the ESP32 firmware design.

Emergency mode handling was one of the standout features observed during testing. Whether activated from the dashboard or via RFID detection, the system switched immediately into priority mode, turning the selected lane Green and all others Red with no delay.

The blue emergency indicator lit up instantly, and once the emergency was cleared, the system resumed the automatic cycle smoothly and safely. No overlapping or contradictory signal states appeared during these transitions, confirming the reliability of the override mechanism.

The web dashboard also showed strong performance across devices, operating smoothly on smartphones, laptops, and tablets. Its lightweight communication structure allowed the interface to respond quickly to user inputs while maintaining continuous real-time status updates. Even during rapid interactions or long open sessions, the dashboard remained stable and visually consistent.

The OLED module performed with equal reliability, presenting clear, real-time system information throughout the testing period. It accurately reflected every phase change, emergency status, and countdown update without flicker or lag. This contributed significantly to system transparency during physical demonstrations.

The hardware components themselves proved durable and dependable. All LEDs responded quickly to GPIO signals, and wiring connections remained stable throughout prolonged use. Power delivery was steady, with no voltage drops or overheating problems. The RFID reader successfully detected authorised tags within its expected range, providing reliable emergency activation throughout the evaluation.

Overall, the system achieved a high level of efficiency, seamlessly handling multiple tasks simultaneously, including LED control, dashboard communication, emergency logic, and OLED updates. Its non-blocking timing design ensured uninterrupted multitasking with no performance degradation. These observations confirm that the system is well-suited for real-time operation and offers a strong foundation for future enhancements and smart-city traffic applications.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 RESULTS

8.1.1 LIVE WEB DASHBOARD CONTROL

The web-based control dashboard performed reliably throughout testing and successfully presented real-time traffic signal information. The interface continuously displayed the active traffic direction whether North, East, or South along with the corresponding signal colour and the remaining countdown for each phase. Users were able to modify signal timings directly from the dashboard, and the system reflected these updates immediately. In addition, the emergency-control buttons provided smooth and instantaneous switching for all three directions, demonstrating that the ESP32 processed remote commands quickly and without delay. Overall, the live dashboard operated seamlessly, offering a highly responsive and intuitive control experience.

Fig 8.1 Smart Traffic Control Panel Interface

In Figure 8.1, the smart traffic control dashboard provides an interface for monitoring and configuring signal timings for multiple directions. It enables real-time adjustment of green, yellow, and red phases through a centralized control panel.

8.1.2 CUSTOM TIMING FEATURE WORKING CORRECTLY

The custom timing functionality was tested extensively to verify its responsiveness and accuracy. When new values were entered for the Green, Yellow, or Red phases such as adjusting

them to 5 seconds, 2 seconds, and 1 second respectively the ESP32 updated its internal timing parameters. In Figure 8.2, These changes took effect within the next phase transition without requiring a system restart or manual intervention. Both individual timing buttons for North, East, and South, as well as the "Apply All" option, operated consistently and as intended.

Fig 8.2 North Lane Timing Updated Successfully.

In Figure 8.2, the smart traffic control dashboard provides an interface for monitoring and configuring signal timings for multiple directions, enabling real-time adjustment of traffic signal phases.

8.1.3 EMERGENCY AMBULANCE MODE

The emergency-priority feature was rigorously tested through the dashboard controls, which provide dedicated buttons for triggering ambulance clearance in the North, East, or South directions, along with a reset option. During evaluation, activating any emergency button immediately shifted the corresponding lane to a Green signal while the remaining directions switched to Red, ensuring an unobstructed passage for the emergency vehicle. The dashboard promptly reflected this change by displaying the status "Emergency Green," confirming successful transition into priority mode

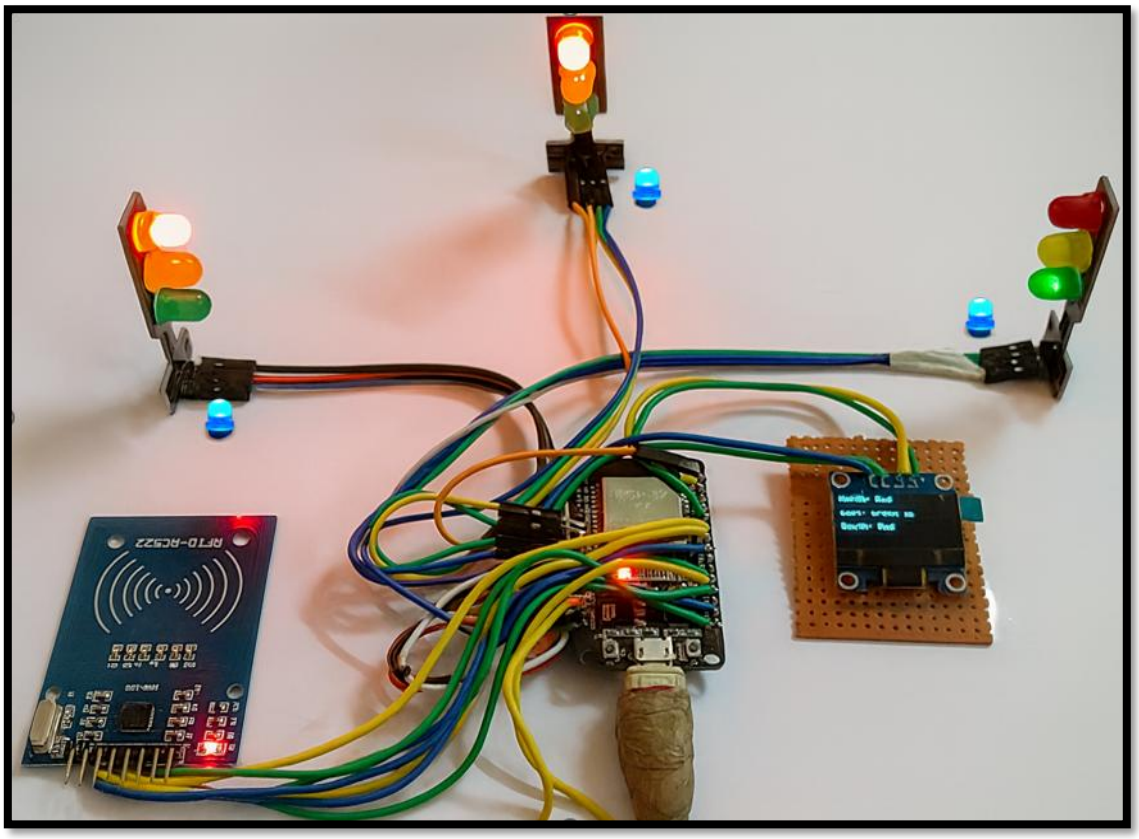

Fig 8.3 Emergency Mode Activated South Lane Yellow

In Figure 8.3, the experimental setup of the ESP32-based traffic control prototype is illustrated with interconnected signal LEDs, an RFID reader, and an OLED display, demonstrating real-time traffic signal operation and emergency handling.

8.1.4 PHYSICAL PROTOTYPE OUTPUT

The physical prototype performed exactly as expected, as confirmed by the hardware observations. In the captured output, the North and South lanes are shown with their Red signals illuminated, while the East lane correctly displays a Green light, matching the system's active direction during testing.

The OLED display reinforces this behaviour by presenting the live status for each lane North: Red, East: Green, and South: Red along with the RFID module visibly powered and operational. The ESP32 executed the traffic control logic continuously without interruption, resets, or timing mismatches. The LED signal states observed on the prototype were fully synchronized with the information shown on the web dashboard, indicating seamless integration between the firmware, user interface, and physical components. all output layers traffic LEDs, OLED display, and dashboard highlights the stability, accuracy, and reliability of the complete system. Overall, the physical prototype successfully reflects the intended system design and validates the effectiveness of the integrated IoT-based traffic control architecture .that the integrated system is functioning reliably and as designed.

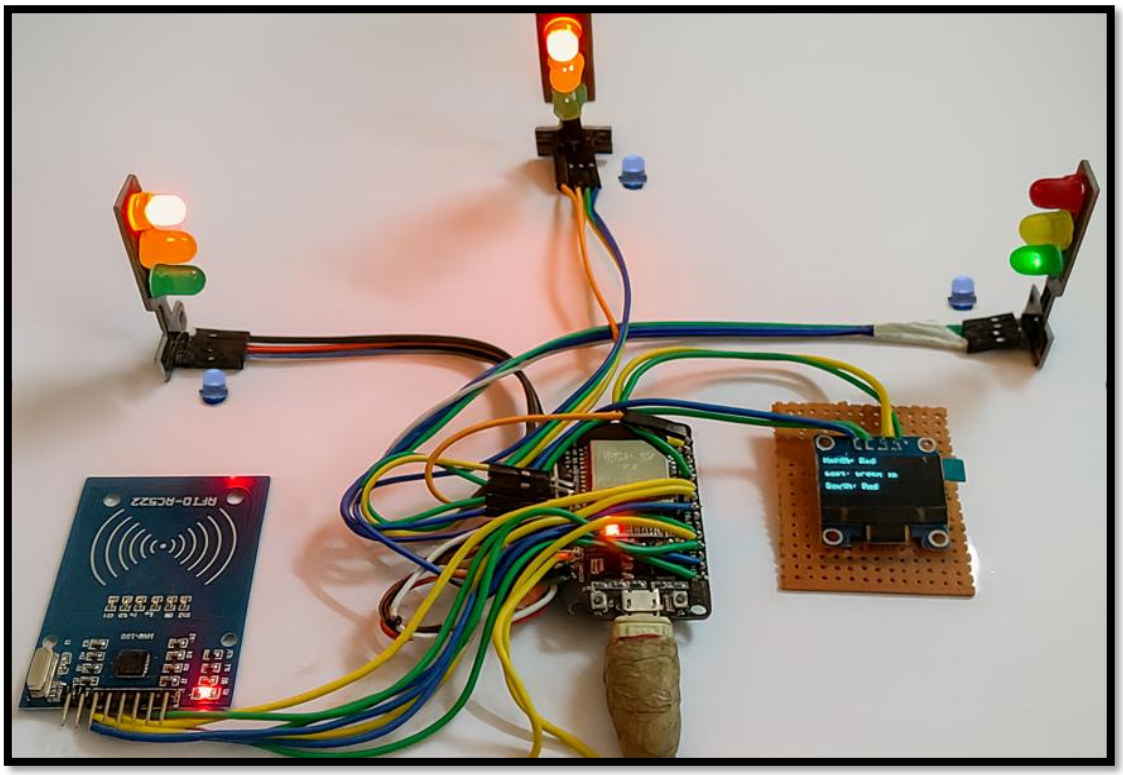

Fig 8.4 Working Prototype with Active Signal Output

In Figure 8.4, the experimental setup of the ESP32-based traffic control prototype is illustrated with connected signal LEDs, an RFID reader, and an OLED display, demonstrating real-time signal control and emergency indication.

8.1.5 OLED DISPLAY RESULT

The OLED display performed accurately throughout testing, showing the active traffic direction, its current colour state, and the status of the remaining directions. In Figure 8.5 During emergency activation, the screen immediately switched to display the emergency alert, confirming real-time responsiveness. The results verify that the OLED provides reliable on-device monitoring without any delay.

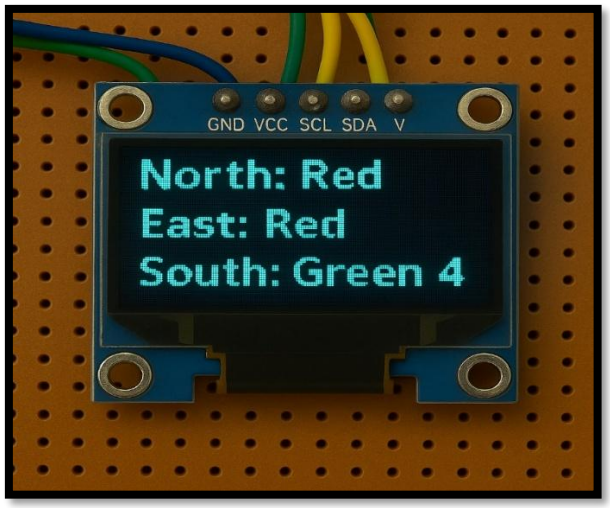

Fig 8.5 Status Display of Each Signals

In Figure 8.5, the OLED display presents the real-time status of traffic signals for multiple directions. It provides clear visual feedback on signal states and remaining phase duration.

8.2 DISCUSSION

8.2.1 OVERVIEW OF OUTCOME ANALYSIS

The IoT-Based Emergency Priority Traffic System showed consistently strong and reliable performance throughout all stages of testing. The automatic Red–Yellow–Green sequence transitioned smoothly for every monitored direction, and the LED indicators remained in perfect harmony with the displayed values on the dashboard. Whenever the software shifted to a new signal phase, the hardware responded instantly, demonstrating excellent coordination between the programmed logic and physical output.

The OLED display further reinforced system reliability by continuously presenting the active lane, current light status, and countdown timing. This ensured that the system's behaviour could be observed directly from the hardware setup, even when the dashboard was not in use.

One of the most effective features observed during testing was the emergency-priority mechanism. Whether activated through the dashboard or triggered by an RFID tag, the ESP32 immediately assigned Green to the priority lane and switched all remaining directions to Red. The illumination of the blue emergency LED provided a clear indication that override mode was engaged. Once the emergency condition was cleared, the controller resumed its normal cycle without any malfunction, confirming that the override logic is both rapid and dependable.

8.2.2 COMPARISON WITH EXISTING TRAFFIC SYSTEMS

Most traditional traffic management systems still rely on fixed-timing mechanisms that operate the same way regardless of actual road conditions. Because they lack automation, these systems cannot adjust signal durations dynamically and offer no built-in method for giving emergency vehicles immediate right-of-way. Any modification to the signal pattern generally requires a technician to make changes manually at the site, and the absence of real-time monitoring means operators have limited visibility into the system's current state or performance.

The proposed IoT-based solution overcomes these limitations by incorporating intelligent and flexible features that improve both responsiveness and overall traffic flow. Through a browser-based dashboard, operators can remotely monitor signal status, modify timing values, and activate emergency overrides without needing physical access to the

junction. The addition of an OLED display provides continuous on-device feedback, enhancing visibility for on-site personnel.

The system also supports both RFID-triggered and manual emergency clearance, giving emergency vehicles immediate passage in critical situations. Because it is built using inexpensive and energy-efficient hardware, the design remains cost-effective while delivering much greater adaptability and reliability than conventional fixed-time controllers.

8.2.3 LIMITATIONS

Although the system performed reliably throughout testing, a few constraints became evident during evaluation. The current prototype is designed for a three-way intersection rather than a conventional four-way junction. While the design can be expanded with additional GPIO pins, the present setup limits real-world applicability. Another limitation relates to the system's dependence on Wi-Fi for dashboard access; although the ESP32 continues running the automatic cycle without interruption, remote monitoring and control cannot function during network outages.

The RFID module used in the project also presents a practical constraint, as its short detection range is appropriate for prototype demonstrations but inadequate for emergency vehicle detection over larger distances in real traffic environments.

The system does not incorporate automated traffic density measurement. Instead, lane congestion must be simulated or manually adjusted, since sensors or camera-based detection were not included in this version. The prototype also lacks cloud integration and long-term data storage, meaning that historical performance, analytics, or multi-junction coordination cannot be supported.

CHAPTER 9

CONCLUSION & FUTURE SCOPE

9.1 CONCLUSION

9.1.1 SUMMARY OF ACHIEVEMENTS

The project successfully developed a complete IoT-Based Emergency Priority Traffic System built around the ESP32 microcontroller. The controller was able to automate the Red–Yellow–Green signal cycle across three traffic directions using non-blocking timing logic, resulting in smooth and uninterrupted phase transitions. A Wi-Fi-enabled dashboard was implemented to allow users to remotely observe the traffic state, modify signal durations, and activate emergency priority whenever required.

The integration of an RFID module added automated emergency detection capability, enabling the system to immediately recognize authorized vehicles and grant them right-of-way without manual input. The OLED display continuously presented live updates of the operational state, including the active lane, current signal colour, phase countdown, and emergency alerts. Through the combined performance of these modules, the project demonstrated a practical and efficient alternative to fixed-timer traffic systems and highlighted how IoT technologies can create more flexible, responsive, and modernized approaches to traffic management.

9.1.2 KEY FINDINGS

The findings of this project show that the ESP32 is capable of handling several critical operations simultaneously, including traffic cycling, Wi-Fi communication, OLED display updates, and emergency-priority execution, without noticeable delays or system slowdown. Throughout long-duration testing, the hardware LEDs, OLED display, and dashboard indicators remained synchronized, reflecting the accuracy of the internal signal-control logic.

Emergency-priority activation, whether triggered by the dashboard or via the RFID module, occurred instantly, confirming the reliability of the priority-override mechanism. The browser-based interface also proved highly effective, offering convenient access to all major controls and eliminating the need for physical adjustments at the traffic junction. These results highlight the system's strong multitasking ability, reliable hardware–software coordination, and suitability for real-time smart-traffic applications.

9.1.3 LIMITATIONS OF THE CURRENT SYSTEM

Although the prototype performed reliably, several constraints were identified. The present system manages only three directions, and extending it to a full four-way intersection would require additional GPIO resources or multiplexing techniques. Traffic density remains a

manually adjustable parameter because no automated sensors or vision-based detection mechanisms have been integrated.

The RFID module's short detection range limits its use to close-range demonstrations rather than real-world intersection placement. The dashboard depends on Wi-Fi availability, meaning remote control is temporarily lost during network interruptions even though the ESP32 continues operating locally. Additionally, the system does not include cloud connectivity or data-logging features, restricting its ability to record long-term traffic information. The prototype also works as a standalone junction without any coordination with nearby intersections. Despite these limitations, it effectively showcases the feasibility and advantages of deploying IoT-based traffic priority systems.

9.2 FUTURE SCOPE

9.2.1 FUNCTIONAL ENHANCEMENTS

Future improvements to the system can focus on expanding its ability to react intelligently to real traffic conditions. One major enhancement involves integrating real-time traffic detection using sensors such as infrared, ultrasonic, or magnetic loop detectors. These inputs would allow the controller to adjust signal timings dynamically instead of relying on manual configuration. Another significant upgrade is the shift from RFID-based emergency recognition to more advanced methods such as Vehicle-to-Infrastructure (V2I) communication, enabling emergency vehicles to be detected from longer distances with higher accuracy.

The system can also be strengthened by incorporating AI or machine-learning algorithms capable of predicting congestion patterns and automatically regulating signal phases. For larger-scale applications, coordinated multi-junction communication could be added, allowing multiple intersections to work together to form efficient "green corridors" during peak traffic hours.

9.2.2 IMPROVEMENTS TO NON-FUNCTIONAL ASPECTS

Enhancing non-functional capabilities will further improve the reliability and usability of the system. Cloud platforms such as Firebase, AWS IoT, or Things Board may be integrated to support centralized monitoring, analytics, and remote firmware updates. Strengthening system security through encrypted communication, user authentication, and controlled access will ensure safe operation, especially when deployed in public environments. Energy efficiency can also be improved by adopting solar-powered units and utilizing low-power modes of the ESP32 to reduce consumption during idle periods. System resilience may be increased through the addition of watchdog timers, redundant timing logic, and automatic fail-safe recovery

strategies, ensuring uninterrupted performance even in the presence of hardware or communication faults.

9.2.3 BROADER PROJECT EXPANSION

On a broader scale, the project offers significant potential for expansion. A dedicated mobile application could be developed to allow operators, emergency responders, and authorized personnel to control or monitor intersections remotely with greater convenience.

The system could also be extended to collect and visualize historical traffic data, enabling authorities to study long-term patterns, evaluate congestion trends, and make informed infrastructural decisions. Further integration into municipal smart-city networks would allow multiple intersections to communicate with each other, facilitating unified traffic governance and citywide optimization of vehicle flow.

REFERENCE

[1]. Smart Traffic Management System with Emergency Vehicle Prioritization and Stolen Vehicle Detection (2024) by Abhirami Js, Dinesh Kumar M, Prasanth S, Dalbert Mario J. Link:https://www.ijert.org/smart-traffic-management-system-with-emergency-vehicle-prioritization-and-stolen-vehicle-detection-using-arduino-technology

[2]. An Optimal Control Strategy for Emergency Vehicle Priority System in Smart Cities Using Edge Computing and IoT Sensors (2023) by R. Pradeep, M. Ramesh, P. Suresh, et al. Link: https://www.sciencedirect.com/science/article/pii/S2665917423000338

[3]. IoT-Based Traffic Management System Prioritizing Emergency Vehicles (2023) by A. Sharma, R. Kumar, S. Gupta. Link:https://www.ijert.org/iot-based-traffic-management-system-prioritizing-emergency-vehicles

[4]. IoT-Based Emergency and Theft Vehicle Identification in Traffic System Using RFID (2024) by Velmurugan V., Arunkumar B., Deepika B., Dhanasree M., Kumaram S. Link:https://ijireeice.com/papers/iot-based-emergency-and-theft-vehicle-identification-in-traffic-system-using-rfid/

[5]. RFID-Based Traffic Control System for Emergency Vehicle (2023) by Gowtham M., Adhwanth D., Manojkumar G., Naveen R., Nazeem K. I. Link: https://ijetms.in/Vol-7-issue-3/Vol-7-Issue-3-34.html